



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Progettazione e Sviluppo di un Tool per l'Estrazione Automatizzata e l'Analisi di Scenari in Simulazioni CARLA e BeamNG per Sistemi ADAS/ADS

RELATORE

Prof. Carmine Gravino

Dott. Antonio Trovato

Università degli Studi di Salerno

CANDIDATO

Mario Celzo

Matricola: 0512118665

Anno Accademico 2024-2025

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

"For all the kids out there who dream the impossible"

-Lewis Hamilton

Abstract

L'evoluzione dei Sistemi Avanzati di Assistenza alla Guida (ADAS) e della guida autonoma (ADS) sta trasformando il settore automobilistico, abilitando funzionalità come frenata automatica d'emergenza, mantenimento di corsia e controllo adattivo della velocità. La validazione di tali sistemi richiede test su un ampio spettro di condizioni operative, incluse situazioni rare o critiche. La sperimentazione su strada, pur necessaria in fase finale, presenta limiti rilevanti: costi elevati, rischi per la sicurezza e difficoltà nel riprodurre condizioni estreme o poco frequenti. Per affrontare queste criticità si è adottato l'uso estensivo della simulazione. Tra i simulatori open-source più diffusi figurano **CARLA**, basato su Unreal Engine e con supporto a scenari Open-SCENARIO (`.xosc`) e a script Python, e **BeamNG.tech**, apprezzato per la fedeltà della dinamica veicolare e il realismo delle collisioni. All'interno di questo lavoro è stato progettato e sviluppato l'*ADAS/ADS Scenario Analysis Tool* (ASAT), un software modulare che automatizza l'esecuzione e l'analisi di scenari di test in CARLA e BeamNG. ASAT esegue scenari descritti sia in formato `.xosc` sia tramite script Python e raccoglie dati strutturati per calcolare tre metriche fondamentali: *Execution Time* (tempo fino a collisione o timeout), *Criticality Score* (indice di rischio guidato da eventi, come collisioni e violazioni) e *Diversity Score* (dissimilarità tra scenari per selezioni non ridondanti). Le uscite in `.json` alimentano processi di ottimizzazione della selezione, riducendo test poco informativi e concentrando l'attenzione su quelli più utili. La compatibilità multi-simulatore è ottenuta tramite un'architettura a componenti e strumenti di parsing/conversione che preservano la semantica degli scenari. Il contributo principale della tesi è un *tool* versatile e riutilizzabile che consente di generare dataset sintetici di scenari critici e di supportare, in modo cost-effective, la ricerca e l'industria nell'ambito della validazione dei sistemi ADAS.

Indice

Elenco delle Figure	iv
Elenco delle Tabelle	v
1 Introduzione	1
1.1 Contesto applicativo	1
1.2 Motivazione dell'uso dei simulatori	3
1.3 Limiti attuali e sfide del contesto	4
1.4 Obiettivi del lavoro	6
1.5 Contributo della tesi	7
2 I simulatori di guida: CARLA e BeamNG	10
2.1 ADAS e ADS: definizioni e differenze	10
2.1.1 La co-simulazione	11
2.2 CARLA	12
2.2.1 Scenari in formato OpenSCENARIO	12
2.2.2 Scenari in formato Python (.py)	13
2.3 BeamNG.tech	15
2.4 Confronto tra i due simulatori	16
3 Metodologia	18

3.1	Struttura del progetto	18
3.2	Fase preliminare	23
3.2.1	Generazione e parsing degli scenari	23
3.2.2	Mappatura degli elementi OpenSCENARIO nel convertitore	25
3.3	Prima fase	26
3.3.1	Script sviluppati per la prima parte	27
3.3.2	Raccolta dei risultati per la prima parte	28
3.4	BehaviorAgent e gli ADAS	29
3.5	Seconda fase	30
3.5.1	Gestione delle esecuzioni	30
3.5.2	Raccolta dei dati nella seconda parte	31
4	Implementazione	33
4.1	Architettura del Sistema	33
4.1.1	La creazione del parser	33
4.1.2	La struttura del parser	34
4.2	Seconda parte: simulazioni e script Python	36
4.2.1	Simulazioni automatiche, gestione del traffico e raccolta dei risultati	36
4.3	Limiti e difficoltà	43
5	Estrazione delle Features	45
5.1	Architettura del Sistema di Estrazione	45
5.1.1	Estrazione da OpenSCENARIO	46
5.1.2	Conversione tra Formati Simulatore	46
5.2	Domini di Informazione Estratti	47
5.2.1	Proprietà Cinematiche e Dinamiche	47
5.2.2	Caratterizzazione degli Agenti	48
5.2.3	Condizioni Meteorologiche	49
5.2.4	Topologia Stradale	50
5.2.5	Eventi Dinamici e Interazioni	51
5.3	Formato di Output e Struttura dei Dati	52
5.3.1	Schema dei Dati di Evento	52

5.3.2	Aggregazione Multi-Scenario	53
5.4	Metriche di Qualità degli Scenari	54
5.4.1	Execution Time	54
5.4.2	Criticality Score	54
5.4.3	Diversity Score	55
5.5	Gestione dei Dati e Tracciabilità	56
5.5.1	Organizzazione della Directory di Output	56
5.5.2	Logging Multi-Livello	57
5.5.3	Gestione Errori e Robustezza	57
5.6	Integrazione con Pipeline di Ottimizzazione	58
6	Conclusioni e Sviluppi Futuri	59
6.1	Sintesi del Lavoro Svolto	59
6.2	Risultati Conseguiti e Contributi Originali	60
6.3	Limitazioni e Criticità	60
6.4	Sviluppi Futuri	61
6.5	Considerazioni Finali	62
	Bibliografia	63

Elenco delle figure

3.1	Pipeline del convertitore da OpenSCENARIO (.xosc) per CARLA a JSON eseguibile in BeamNG: parsing, normalizzazione, mapping e generazione dell'output.	24
-----	---	----

Elenco delle tabelle

2.1	Confronto sintetico tra CARLA e BeamNG.tech	16
4.1	Campi chiave dei file JSON generati dagli scenari	41

CAPITOLO 1

Introduzione

1.1 Contesto applicativo

Negli ultimi anni, il settore della guida autonoma ha conosciuto uno sviluppo estremamente rapido, spinto dalla crescente domanda di veicoli intelligenti, sicuri ed efficienti [1, 2, 3]. Al centro di questa evoluzione si trovano i **Sistemi Avanzati di Assistenza alla Guida** (ADAS), ovvero un insieme di tecnologie integrate nei veicoli moderni che supportano il conducente nella prevenzione e mitigazione degli incidenti [2, 4]. Tra le funzionalità più diffuse rientrano la frenata automatica di emergenza (*Automatic Emergency Braking*), il mantenimento di corsia (*Lane Keeping Assist*), il controllo adattivo della velocità (*Adaptive Cruise Control*) e il riconoscimento della segnaletica stradale [5, 3]. L'efficacia di tali sistemi non dipende soltanto dall'accuratezza degli algoritmi di percezione e decisione, ma anche dalla loro capacità di gestire correttamente scenari complessi, eterogenei e, in particolare, **situazioni critiche** che potrebbero mettere a rischio l'incolumità degli utenti della strada [6, 2, 4]. Per poter garantire prestazioni affidabili, è necessario sottoporre gli ADAS a un numero molto elevato di test che includano diversi tipi di scenari: scenari **ordinari**, come il traffico regolare o la guida in autostrada; scenari **rari**, che comprendono, ad esempio, manovre impreviste di altri veicoli o attraversamenti improvvisi di pedoni [2, 3];

e scenari **critici**, caratterizzati da collisioni imminenti, condizioni meteorologiche avverse o intersezioni congestionate [5, 7, 4]. Nel contesto della validazione, uno **scenario di test** può essere definito come un insieme di condizioni ambientali, configurazioni del traffico e comportamenti dei partecipanti (veicoli, pedoni, ciclisti) in cui il sistema ADAS viene valutato [8, 9, 10]. In letteratura, la necessità di disporre di grandi insiemi di scenari di test è stata analizzata in modo sistematico, evidenziando come la sola distinzione tra scenari ordinari e rari non sia sufficiente a garantire una validazione robusta dei sistemi di guida autonoma [4, 3]. In una revisione della letteratura dedicata agli *scenari critici*, gli autori propongono una tassonomia dei metodi di identificazione basati su dati reali, modelli formali e tecniche di ottimizzazione, mostrando che la capacità di mettere alla prova il sistema in condizioni prossime al fallimento è un prerequisito per una stima affidabile del rischio residuo [4]. Queste considerazioni si allineano con gli obiettivi di questa tesi, che mira a fornire uno strumento in grado di eseguire e analizzare automaticamente scenari potenzialmente critici, mettendo a disposizione metriche strutturate utili sia per la validazione sia per future strategie di selezione automatica dei test [7, 1]. La sperimentazione di tali scenari su strada reale, sebbene fondamentale per la fase finale di validazione, presenta tuttavia diversi svantaggi [3, 11]. Tra i principali si possono evidenziare: (i) i costi molto elevati legati all'impiego di veicoli, personale e infrastrutture dedicate [6]; (ii) le difficoltà logistiche e legali nell'eseguire situazioni ad alto rischio in contesti reali [11]; (iii) la scarsa possibilità di riprodurre fedelmente eventi rari o complessi, come collisioni o condizioni meteorologiche estreme [4, 3]; e (iv) le implicazioni etiche e di sicurezza che coinvolgono direttamente i partecipanti umani [11]. Per tali ragioni, l'impiego di **simulatori di guida** è diventato una strategia chiave nella fase di sviluppo e pre-validazione dei sistemi ADAS, consentendo di esplorare un ampio spettro di scenari in modo controllato e senza rischi per l'utenza reale [8, 12]. Gli scenari possono essere descritti in diversi formati. Tra questi, l'**OpenSCENARIO** (`.xosc`) rappresenta uno standard aperto basato su XML, ampiamente utilizzato per la definizione strutturata di scenari complessi, includendo condizioni iniziali, attori coinvolti, traiettorie e trigger di eventi [13, 10, 14]. In alternativa, è possibile ricorrere a **script Python** (`.py`), che permettono di definire scenari personalizzati mediante codice, offrendo una maggiore flessibilità e la possibilità di integrare logiche

procedurali utili, ad esempio, per generare comportamenti dinamici e casuali [12, 9].

1.2 Motivazione dell'uso dei simulatori

L'utilizzo di simulatori di guida consente di effettuare test su larga scala, abbattendo drasticamente i costi e annullando i rischi fisici per i partecipanti [1, 8, 3]. Attraverso questi strumenti, è possibile ricreare eventi rari o pericolosi, impossibili o estremamente complessi da replicare in strada, con la possibilità di variare parametri ambientali (ad esempio visibilità, aderenza del manto stradale, traffico) in maniera controllata [3, 11]. La capacità di generare scenari sintetici rappresenta un vantaggio fondamentale nella selezione intelligente dei casi di test [1, 2, 4, 15, 16], poiché consente di esporre i sistemi ADAS a condizioni edge-case e corner-case che richiederebbero milioni di chilometri di guida reale per essere osservate naturalmente, riducendo i costi di validazione senza compromettere la copertura del dominio operativo [5, 12, 9]. Nel contesto di questa tesi, sono stati considerati due tra i simulatori open-source più rilevanti nel panorama automotive: **CARLA** (Car Learning to Act), sviluppato con Unreal Engine, dotato di un'ampia libreria di modelli urbani e rurali, supporto nativo a scenari in formato OpenSCENARIO (`.xosc`) e a script Python personalizzati, oltre a un sistema avanzato di gestione del traffico autonomo [8, 17, 7]; e **BeamNG.tech**, riconosciuto per la simulazione fisica estremamente accurata e la fedeltà nella modellazione delle dinamiche veicolari [18, 12], in particolare nelle collisioni [12, 9]. La combinazione di questi strumenti risulta particolarmente vantaggiosa: CARLA offre un ecosistema ricco per la gestione e generazione di scenari di traffico [17, 2], mentre BeamNG garantisce un livello di realismo fisico utile per analisi post-incidente e studi sull'impatto [18, 12, 9]. In questo progetto è stato impiegato operativamente CARLA (versione 0.9.15) per l'esecuzione automatizzata degli scenari e l'estrazione dei dati, ma l'architettura del tool sviluppato è modulare, così da poter essere estesa in futuro anche a BeamNG o ad altri simulatori emergenti [12, 7].

Questo approccio fornisce un contributo concreto sia alla ricerca che all'industria, in quanto:

- permette di generare **dataset sintetici** utili per l'addestramento e la validazione

di algoritmi ADAS [8, 7, 9];

- supporta metodologie di **test selection**, selezionando scenari ad alto valore informativo per ridurre tempi e costi di validazione [7, 1, 6, 15];
- facilita studi di **safety assurance** senza rischi per l'ambiente reale [3, 2, 4].

1.3 Limiti attuali e sfide del contesto

Sebbene i simulatori di guida rappresentino oggi uno strumento imprescindibile per lo sviluppo e la validazione dei **Sistemi Avanzati di Assistenza alla Guida** (ADAS), la loro adozione su larga scala è tutt'altro che priva di criticità [1, 2, 3, 4]. Infatti, affinché un test in simulazione possa essere considerato efficace, è necessario che gli scenari riproducano condizioni quanto più possibile vicine a quelle reali, che siano vari e diversificati, e che le metriche di valutazione siano ben definite e coerenti con gli obiettivi di sicurezza e prestazioni del sistema [5, 3]. Tuttavia, allo stato attuale, l'uso dei simulatori presenta alcune limitazioni chiave. Una prima criticità riguarda la **scarsa disponibilità di dataset realistici**, soprattutto in formato **OpenSCENARIO** [13, 10, 14]. Sebbene esistano librerie pubbliche e repository comunitari, questi spesso contengono scenari sintetici generici, con un livello di dettaglio insufficiente per testare in maniera approfondita situazioni ad alto rischio [6, 2, 4, 16]. Inoltre, molte raccolte non includono parametri essenziali come condizioni meteorologiche realistiche, comportamento umano dei pedoni [2, 11], o dinamiche di traffico non deterministiche [5, 12], che sono fondamentali per la valutazione dei sistemi ADAS avanzati [3]. Un'ulteriore limitazione è rappresentata dalla difficoltà nell'automatizzare completamente la raccolta e l'analisi dei dati durante le simulazioni [7, 1]. In molti simulatori, infatti, le informazioni generate dagli scenari — come eventi di collisione, superamenti di velocità o tempi di reazione — non vengono registrate in maniera strutturata oppure richiedono l'implementazione di script personalizzati complessi [7, 9]. Questo comporta un elevato sforzo di integrazione, soprattutto quando si desidera correlare più metriche (ad esempio tempo di esecuzione, criticità e diversità) in un unico processo automatizzato [7, 6]. Un ulteriore ostacolo è dato dalla **limitata interoperabilità tra simulatori diversi** [5, 12, 4, 19]. Nonostante l'e-

sistenza di standard aperti come OpenSCENARIO [13, 10, 14] e OpenDRIVE, ogni simulatore implementa solo un sottoinsieme delle specifiche o aggiunge estensioni proprietarie poco documentate [5, 19]. Ne consegue che un file `.xosc` eseguibile in CARLA potrebbe non risultare compatibile con BeamNG.tech senza adattamenti manuali, rendendo difficile lo sviluppo di strumenti di test universali e limitando la possibilità di confrontare direttamente le prestazioni di un sistema su più piattaforme [12, 9]. Infine, si segnala l'**assenza di strumenti che combinino esecuzione automatica e valutazione intelligente degli scenari** [1, 7]. Molte piattaforme consentono l'esecuzione sequenziale degli scenari, ma poche integrano meccanismi di **test selection** basati su metriche di copertura o sul valore informativo dei test [1, 6]. In mancanza di tali strumenti, si rischia di eseguire scenari ridondanti o poco rilevanti, con conseguente aumento dei costi computazionali e riduzione dell'efficienza complessiva del processo di validazione [7, 4]. L'analisi critica del contesto trova riscontro anche in diversi studi recenti, che mostrano come una parte consistente del budget di test in simulazione venga spesa in scenari poco informativi o ridondanti, con benefici limitati in termini di copertura effettiva dei casi di rischio [4, 6, 15]. Alcune tesi e lavori applicativi nel dominio dei veicoli autonomi propongono già pipeline complesse di validazione basate su simulazione, nelle quali la difficoltà principale non è più la semplice generazione di scenari, ma la loro selezione e prioritarizzazione in funzione del valore informativo [11, 7]. In parallelo, sono stati proposti approcci di tipo *search-based testing* in cui algoritmi evolutivi generano automaticamente scenari di guida particolarmente critici, guidati da funzioni di fitness che incorporano indici di rischio e, in alcuni casi, termini di diversità tra scenari, rafforzando ulteriormente la necessità di strumenti come ASAT che forniscano in uscita metriche di criticità e dissimilarità utilizzabili in tali pipeline [20, 21, 9, 22]. Un ulteriore elemento critico riguarda la **rappresentazione della complessità reale** nei simulatori [6, 11]. Sebbene sia possibile modellare condizioni meteo, scenari urbani complessi e comportamenti degli altri veicoli, replicare fedelmente l'imprevedibilità e la variabilità del mondo reale rimane una sfida aperta [6, 3]. Ad esempio, i pedoni simulati tendono a seguire traiettorie e movimenti troppo regolari rispetto a quelli osservabili nel comportamento reale, riducendo così l'imprevedibilità tipica dell'interazione umana [2, 11]. Anche i veicoli non sempre riproducono fedelmente le irregolarità proprie della gui-

da quotidiana, come sorpassi azzardati, frenate improvvise o momenti di distrazione del conducente [2, 4]. A ciò si aggiunge il fatto che le condizioni di illuminazione e visibilità non riescono a replicare pienamente le variazioni improvvise e non lineari che caratterizzano l'ambiente reale, limitando ulteriormente il grado di realismo raggiungibile in simulazione [8, 3]. Queste limitazioni non solo riducono l'affidabilità della validazione in simulazione, ma possono generare **gap prestazionali** quando il sistema viene testato su strada [3, 11]. In altre parole, un ADAS che si comporta correttamente in un ambiente simulato potrebbe non reagire altrettanto bene in condizioni reali, specialmente se gli scenari simulati non hanno coperto la gamma completa di eventi possibili [3, 4]. Alla luce di queste sfide, diventa essenziale sviluppare strumenti che siano in grado, da un lato, di generare e gestire scenari più realistici e diversificati [7, 9, 20] e, dall'altro, di automatizzare l'intero processo di raccolta e strutturazione dei dati di test [1, 7]. È inoltre fondamentale che tali strumenti facilitino la compatibilità tra diversi simulatori, riducendo le barriere tecniche e i costi di adattamento [5, 12]. Infine, un aspetto cruciale consiste nell'integrare criteri intelligenti di selezione degli scenari, così da ottimizzare tempi e risorse senza compromettere la copertura dei casi di test e la qualità complessiva della validazione [1, 6, 4]. Proprio su queste direttrici si colloca il presente lavoro di tesi, che mira a colmare il divario tra esecuzione simulata e ottimizzazione del processo di validazione ADAS, fornendo un **tool modulare e scalabile** capace di operare con scenari sia in formato `.xosc` sia in script Python, e di integrare metriche di analisi mirate a identificare gli scenari a maggiore valore informativo per il miglioramento continuo dei sistemi di guida assistita [7, 1].

1.4 Obiettivi del lavoro

L'obiettivo di questa tesi è la progettazione e lo sviluppo di un **tool automatico per l'estrazione e l'analisi di scenari critici** destinati alla validazione di **Sistemi Avanzati di Assistenza alla Guida (ADAS)** [7, 4]. Tale strumento, denominato **ADAS Scenario Analysis Tool (ASAT)**, è stato concepito come una piattaforma modulare e scalabile, capace di operare in diversi ambienti di simulazione, con particolare

attenzione a CARLA e BeamNG [17, 18, 12]. Il sistema proposto nasce dall'esigenza di disporre di un **processo sistematico e automatizzato** in grado di [1, 7]:

- **Eseguire automaticamente scenari** sviluppati in formato `.py` o, in prospettiva, anche `.xosc`, garantendo un'ampia flessibilità nella definizione delle condizioni iniziali e degli eventi simulati [12, 7, 9].
- **Raccogliere dati strutturati e oggettivi** da ogni sessione di simulazione, organizzandoli in `file JSON` per facilitarne l'analisi, la condivisione e l'integrazione in pipeline di testing più complesse [7, 6].
- **Calcolare, per ogni scenario, tre metriche chiave** che costituiscono il cuore della valutazione [7, 1]:
 - **Execution Time**: tempo effettivo di simulazione fino al verificarsi di una collisione o al raggiungimento di un timeout predefinito, utile per stimare la durata e la complessità operativa dello scenario [7].
 - **Criticality Score**: indice di pericolosità dello scenario, calcolato in base a parametri quali collisioni, manovre rischiose, condizioni meteo avverse o congestione del traffico [7, 6, 4].
 - **Diversity Score**: misura della dissimilarità dello scenario rispetto ad altri già analizzati, strumento essenziale per evitare ridondanze e massimizzare la copertura dei casi di test [6, 9, 20, 22].
- **Supportare la portabilità tra simulatori**, rendendo possibile la conversione di scenari originariamente sviluppati per CARLA in formati compatibili con BeamNG, ampliando così il potenziale di utilizzo dell'ambiente [12, 7, 9].

1.5 Contributo della tesi

Il contributo principale di questo lavoro risiede nello sviluppo di una **pipeline di analisi end-to-end** che consente di passare in modo fluido dalla definizione dello scenario alla sua esecuzione e valutazione automatizzata [7, 1]. Questo approccio permette di generare dataset sintetici di scenari critici, utili sia per il testing tradizionale

sia come dati di addestramento per modelli di machine learning [8, 9], di ottimizzare il processo di validazione riducendo il numero di scenari ridondanti e concentrandosi su quelli con maggior valore informativo [1, 6], e di integrare metodologie di apprendimento automatico per la selezione e la classificazione degli scenari, aprendo la strada a sistemi di testing adattivi e intelligenti [1, 7, 20]. Questo approccio consente di ottimizzare il rapporto tra copertura del dominio operativo e risorse computazionali impiegate, riducendo drasticamente il numero di esecuzioni necessarie per identificare scenari safety-critical [6, 4]. Dal punto di vista operativo, l'ASAT gestisce due diverse modalità di generazione e analisi degli scenari. Da un lato, sono stati sviluppati **scenari casuali in Python**, capaci di introdurre comportamenti variabili e non deterministici, come manovre improvvise dei veicoli o condizioni meteorologiche casuali [12, 7, 9]. L'esecuzione di tali scenari, combinata con l'estrazione automatica delle caratteristiche principali, consente di ottenere un dataset di file JSON contenenti informazioni strutturate [7, 6]. All'interno di questo dataset, alcuni scenari risultano non critici mentre altri includono eventi ad alto rischio; ciò fornisce la base per successivi processi di selezione e ottimizzazione, focalizzati sugli scenari più rilevanti [7, 1, 4]. La generazione stocastica di scenari con parametri variabili rappresenta una strategia efficace per esplorare ampie porzioni dello spazio operativo, aumentando la probabilità di identificare situazioni edge-case e corner-case che sfuggirebbero a una definizione manuale [1, 9, 20, 22, 16]. Dall'altro lato, per quanto riguarda gli **scenari OpenSCENARIO (.xosc)**, è stato sviluppato e utilizzato lo script `run_and_log_scenarios.py`, che esegue lo scenario e ne calcola un indice di criticità sulla base delle caratteristiche statiche e dinamiche osservate [7, 13]. In questo caso, la logica non produce un dataset ampio e randomizzato, ma piuttosto permette di valutare sistematicamente scenari predefiniti, verificandone la coerenza con i requisiti di test [10, 14]. Un aspetto aggiuntivo riguarda il **parser di conversione tra formati**, che consente di trasformare scenari da OpenSCENARIO (.xosc) a BeamNG.tech [12, 7]. Sebbene in questo lavoro il parser non sia stato utilizzato direttamente nella pipeline di analisi (poiché i test si sono concentrati su scenari .py e .xosc eseguiti nativamente), esso rappresenta una funzionalità di rilievo in ottica futura [12, 9]. In particolare, la possibilità di trasferire scenari tra simulatori diversi apre la strada a studi comparativi e a validazioni incrociate su piattaforme eterogenee,

migliorando la generalizzabilità dei risultati [5, 3]. L'interoperabilità tra simulatori diversi costituisce un requisito fondamentale per la validazione cross-platform dei sistemi ADAS [6, 5], consentendo di verificare che le prestazioni del sistema sotto test siano consistenti indipendentemente dall'ambiente di esecuzione e di sfruttare i punti di forza complementari di ciascun simulatore nella caratterizzazione di scenari critici [12, 4]. Dal punto di vista della ricerca, l'ASAT si propone quindi come strumento versatile, estendibile con nuove metriche, adattabile a diversi simulatori e integrabile in pipeline di validazione industriali [7, 1]. Inoltre, fornendo un meccanismo strutturato di raccolta e analisi dati, la soluzione proposta contribuisce a colmare il divario tra simulazione virtuale e sperimentazione reale, ponendo le basi per futuri studi sull'ottimizzazione del testing dei sistemi ADAS [6, 3, 4].

I simulatori di guida: CARLA e BeamNG

In questo capitolo vengono descritti i simulatori di guida **CARLA** e **BeamNG**, due ambienti virtuali di simulazione largamente utilizzati nel contesto della ricerca su veicoli autonomi e sistemi avanzati di assistenza alla guida (ADAS) [2, 5, 17, 18, 8, 9, 3]. L'obiettivo è fornire un quadro tecnico-funzionale di entrambi i simulatori, evidenziando come avvenga la generazione e l'esecuzione di scenari di test realistici [12, 6, 13, 14, 19]. Si evidenzia sin da subito che, pur avendo studiato e analizzato entrambi gli ambienti, il lavoro di tesi è stato svolto **materialmente su CARLA**, mentre BeamNG è stato utilizzato inizialmente nella fase di prototipazione [12, 9].

2.1 ADAS e ADS: definizioni e differenze

I Sistemi Avanzati di Assistenza alla Guida (ADAS, *Advanced Driver Assistance Systems*) rappresentano una classe di tecnologie progettate per supportare il conducente in specifiche situazioni, migliorando sicurezza e comfort [2, 5, 3, 23, 4]. Essi comprendono funzionalità come il mantenimento della corsia, la frenata automatica di emergenza e il cruise control adattivo [2, 3]. In tutti questi casi, il sistema svolge un ruolo di ausilio, ma la responsabilità finale rimane sempre in capo al conducente umano, che mantiene il controllo decisionale [23]. Gli ADAS corrispondono ai livelli SAE

1 e 2 della classificazione internazionale per l'automazione veicolare, dove il sistema fornisce supporto in compiti specifici ma richiede sempre la supervisione attiva del conducente [6, 23, 3]. Gli Autonomous Driving Systems (ADS), invece, costituiscono i livelli più elevati della classificazione SAE (dal livello 3 al livello 5) [2, 23, 4]. Essi sono progettati per assumere il controllo completo del veicolo in contesti definiti, integrando sensori, modelli di percezione, pianificazione e controllo [2, 11]. L'obiettivo degli ADS non è più assistere l'essere umano, ma sostituirlo interamente nelle decisioni di guida. In altre parole, mentre gli ADAS si configurano come strumenti di supporto, gli ADS incarnano la vera e propria guida autonoma [2, 23].

2.1.1 La co-simulazione

Il testing degli ADS si basa sempre più sul concetto di co-simulazione, intesa come l'integrazione di più simulatori eterogenei per rappresentare scenari complessi e realistici [5, 12, 3, 4, 19]. Recenti lavori propongono meta-modelli formali per la definizione strutturata di scenari di test in ambienti co-simulativi, facilitando l'interoperabilità tra piattaforme e la ripetibilità dei test [19]. Un esempio rilevante è dato dall'uso congiunto di *CARLA*, che garantisce una ricca modellazione urbana e un realismo elevato nella gestione del traffico e degli attori, e di *BeamNG.tech*, che offre un modello fisico molto accurato per le dinamiche veicolari e le collisioni [12, 7, 18, 8, 9]. La co-simulazione consente dunque di combinare i punti di forza di ciascun simulatore [5, 3]. Essa permette, da un lato, di testare il sistema in condizioni limite difficilmente riproducibili in un unico ambiente [2, 4, 11]; dall'altro, di validare comportamenti multidominio, come la percezione e la pianificazione in *CARLA*, insieme alla dinamica e all'impatto fisico in *BeamNG* [12, 9]. L'effetto complessivo è un incremento della copertura e della varietà dei test, con un miglior bilanciamento tra realismo e costo computazionale [6, 3]. L'approccio co-simulativo consente inoltre di sfruttare in modo ottimale le risorse computazionali, eseguendo simulazioni fisiche dettagliate solo quando necessario e delegando la generazione di scenari ad alto livello a simulatori meno onerosi dal punto di vista computazionale [6, 7, 4]. In letteratura la co-simulazione viene riconosciuta come una delle strategie più efficaci per migliorare la significatività dei test sugli ADS e, al contempo, ridurne i costi complessivi.

sivi [3, 5, 20, 19]. Studi recenti hanno infatti mostrato come la selezione intelligente dei casi di test, supportata anche da tecniche di apprendimento automatico [1, 7, 21, 22], possa rendere la sperimentazione più sostenibile ed efficiente, riducendo il numero di scenari ridondanti e concentrando le risorse computazionali sugli scenari ad alto valore informativo per la validazione dei requisiti di sicurezza [6, 2, 4].

2.2 CARLA

CARLA (Car Learning to Act) [17] è un simulatore open-source basato su Unreal Engine, progettato per supportare lo sviluppo, l’addestramento e la validazione di sistemi di guida autonoma [8, 12]. Tra le principali funzionalità offerte da CARLA vi sono la possibilità di ricreare ambienti urbani realistici, completi di semafori, segnali stradali, pedoni e altri veicoli [8, 17]; il controllo accurato delle condizioni meteorologiche, della luce e dell’intensità del traffico; e infine l’interfacciamento diretto con agenti di guida, sensori e modelli di rete neurale, che rende possibile testare algoritmi complessi in scenari dinamici e altamente personalizzabili [7]. CARLA si è affermato nella comunità di ricerca come uno degli ambienti simulativi più utilizzati per il testing e la validazione di sistemi ADAS/ADS, grazie alla sua natura open-source, all’elevato livello di realismo visivo e alla disponibilità di API Python per il controllo programmatico [17, 12, 7].

2.2.1 Scenari in formato OpenSCENARIO

CARLA supporta l’esecuzione di scenari attraverso il formato **OpenSCENARIO**, uno standard aperto ampiamente utilizzato nell’industria automobilistica e nella ricerca per la descrizione formale di situazioni di guida [13, 10]. Ogni file `.xosc` è basato su XML e organizza le informazioni secondo una struttura gerarchica che rende gli scenari leggibili, riutilizzabili e interoperabili tra diversi simulatori [5]. In particolare, un file OpenSCENARIO comprende la definizione degli **attori** (come veicoli, pedoni o oggetti statici), le loro **traiettorie** e i **comportamenti** da seguire, nonché i **trigger** che determinano l’inizio, la fine o la transizione tra eventi [13]. A questi elementi si aggiungono le **condizioni ambientali**, come meteo, visibilità o

stato della carreggiata, che permettono di riprodurre un’ampia varietà di contesti di guida [10, 3]. OpenSCENARIO, sviluppato da ASAM (Association for Standardization of Automation and Measuring Systems), rappresenta lo standard de facto per la descrizione formale di scenari dinamici nel settore automotive, consentendo l’interoperabilità tra diverse piattaforme di simulazione e facilitando la condivisione di casi di test tra istituzioni di ricerca e aziende [13, 10]. Per eseguire questi scenari all’interno di CARLA, viene utilizzato lo strumento `ScenarioRunner`, un modulo esterno sviluppato in Python che funge da interprete dei file `.xosc` [7]. `ScenarioRunner` traduce la descrizione formale dello scenario in azioni concrete all’interno della simulazione, controllando in maniera sincronizzata i diversi attori e monitorando l’evoluzione delle condizioni specificate [7]. Questo approccio consente di riprodurre scenari complessi in modo indipendente dall’implementazione del simulatore, favorendo così la comparabilità dei risultati e l’integrazione con pipeline di validazione già esistenti [5, 7]. L’adozione di OpenSCENARIO offre dunque il vantaggio di una rappresentazione **standardizzata e interoperabile**, che garantisce coerenza tra diversi ambienti e piattaforme di test [13]. Tuttavia, come vedremo, tale rigidità strutturale può diventare anche un limite quando si desidera introdurre variabilità dinamica o comportamenti altamente personalizzati, motivo per cui in questo lavoro è stato scelto di privilegiare gli scenari programmabili in Python [12].

2.2.2 Scenari in formato Python (.py)

Oltre agli scenari in formato `.xosc`, che seguono lo standard *OpenSCENARIO* e descrivono in maniera dichiarativa lo svolgimento di una simulazione, la piattaforma CARLA consente la definizione di scenari attraverso script Python [12, 13, 14, 8]. Questi scenari sono implementati sfruttando direttamente la **Python API di CARLA**, offrendo quindi un controllo procedurale e dinamico sul comportamento degli attori, sulle condizioni ambientali e sugli eventi che caratterizzano la simulazione [12, 7, 17, 9]. La natura programmabile degli scenari in `.py` consente di introdurre logiche condizionali complesse, come ad esempio cambiamenti di traiettoria in base alla distanza dal veicolo ego; di simulare comportamenti imprevedibili o stocastici, quali frenate improvvise o attraversamenti pedonali casuali [7, 9, 20]; di generare

dinamicamente attori come veicoli, pedoni e ostacoli durante l'esecuzione [9, 8]; e persino di modificare in tempo reale le condizioni ambientali e del traffico [12, 7, 17]. Questa flessibilità rende gli scenari Python particolarmente adatti a testare situazioni ad **alta variabilità e criticità**, difficilmente descrivibili in un formato statico come l'.xosc, il quale, pur essendo standardizzato e interoperabile, impone una struttura più rigida e meno adatta a scenari fortemente dinamici [7, 13, 14, 16]. Nel contesto di questo lavoro di tesi, l'uso degli scenari .py è stato preferito per diverse ragioni [12, 7]. In primo luogo, essi offrono una maggiore espressività e personalizzazione, poiché la programmazione diretta permette di definire scenari complessi senza le limitazioni sintattiche di OpenSCENARIO [7, 9, 20]. Inoltre, risultano perfettamente integrabili con il tool sviluppato, che è in grado di lanciare automaticamente un elevato numero di scenari Python in sequenza, raccogliendo i dati delle simulazioni in maniera strutturata [7, 6]. Un ulteriore vantaggio è rappresentato dalla possibilità di introdurre variabilità, mantenendo invariato lo stesso script di base [12, 9, 21]. La capacità di generare varianti parametriche di uno stesso scenario base è cruciale per la creazione di dataset sintetici diversificati, utilizzabili sia per il testing diretto che come input per algoritmi di test selection basati su machine learning [1, 7, 9, 4], che richiedono ampie collezioni di scenari annotati per apprendere pattern di criticità e rilevanza [1, 6]. Infine, la rapidità di prototipazione rende gli scenari in Python particolarmente adatti, poiché la loro implementazione riduce significativamente i tempi di sviluppo e di debugging rispetto alla creazione di scenari complessi in .xosc [12, 9].

Gli scenari selezionati includono esempi provenienti sia dai test ufficiali di CARLA sia da repository pubblici, opportunamente adattati alle esigenze del progetto. Alcuni di essi sono:

- `follow_leading_vehicle.py` — valutazione del comportamento del veicolo ego nel seguire un veicolo leader in diverse condizioni.
- `cross_traffic.py` — simulazione di traffico trasversale in incroci non semaforizzati.
- `dynamic_obstacle.py` — gestione di ostacoli che compaiono o si muovono improvvisamente.

- `adversarial_npc.py` — veicoli avversari che eseguono manovre imprevedibili.

In sintesi, gli scenari Python si sono dimostrati fondamentali per esplorare un ampio spettro di situazioni complesse e realistiche, fornendo al **ADAS Scenario Analysis Tool (ASAT)** un insieme di test ricco e variegato, ideale per valutare criticità, tempi di reazione e diversità delle situazioni simulate [7]. L'approccio procedurale alla generazione di scenari si allinea con le strategie di testing cost-effective [6, 9], che enfatizzano l'importanza di massimizzare la copertura del dominio operativo minimizzando il numero di esecuzioni ridondanti, obiettivo raggiungibile attraverso la variazione sistematica dei parametri di scenario e la successiva selezione intelligente dei casi più informativi [7].

2.3 BeamNG.tech

BeamNG.tech [18, 12, 9] è la versione per ricerca del simulatore BeamNG, basato su un motore fisico avanzato capace di simulare danni realistici ai veicoli, dinamiche di guida dettagliate e ambienti estesi [18, 12]. BeamNG si distingue per la sua capacità di offrire una simulazione avanzata della fisica del veicolo, includendo aspetti come il comportamento delle sospensioni, le deformazioni della carrozzeria e la gestione degli attriti [12, 9, 3]. L'interazione con l'ambiente simulato è resa possibile attraverso le API Python fornite dal modulo `beamngpy`, che consente agli sviluppatori di controllare con precisione i veicoli, generare dinamicamente il traffico e accedere a dati dettagliati in tempo reale [18, 12, 7]. Queste caratteristiche rendono BeamNG uno strumento particolarmente adatto per studi che richiedono un alto grado di realismo fisico e una gestione accurata delle dinamiche di guida [12, 9, 20].

2.4 Confronto tra i due simulatori

Caratteristica	CARLA	BeamNG.tech
Motore grafico	Unreal Engine	Torque3D
Fisica veicoli	Semplificata	Reale e deformabile
Supporto OpenSCENARIO	Completo	Limitato (non nativo)
Interfaccia API Python	Sì, ma complessa	Sì, tramite beamngpy
Supporto macOS	Assente	Limitato, ma gestibile
Stabilità	Buona ma instabile in alcuni scenari	Elevata

Tabella 2.1: Confronto sintetico tra CARLA e BeamNG.tech

La tabella precedente evidenzia in maniera schematica le differenze principali tra i due simulatori, ma per comprendere appieno le loro potenzialità è necessario analizzare in modo più dettagliato ciascun aspetto [12, 3]. Per quanto riguarda il **motore grafico**, CARLA si basa su Unreal Engine, una piattaforma ampiamente utilizzata nell'industria videoludica che garantisce un elevato realismo visivo, condizioni di illuminazione dinamiche e scenari urbani dettagliati [8, 17]. BeamNG.tech, invece, utilizza Torque3D, che pur offrendo una grafica meno sofisticata, è ottimizzato per l'integrazione con il proprio motore fisico e garantisce prestazioni più stabili durante simulazioni ad alta intensità computazionale [18, 12, 9]. Sul piano della **fisica veicolare**, la differenza è ancora più marcata: CARLA adotta un modello semplificato, utile per simulazioni di traffico su larga scala, ma meno accurato nella rappresentazione delle dinamiche reali del veicolo [8, 3]. Al contrario, BeamNG si distingue per il suo approccio *soft-body*, in grado di simulare in tempo reale deformazioni, attriti e forze meccaniche complesse, rendendolo ideale per analisi di impatto e validazione di scenari legati alla sicurezza passiva [12, 18, 9]. Relativamente al **supporto OpenSCENARIO**, CARLA garantisce una compatibilità pressoché completa con lo standard, rendendo più immediata la definizione di scenari secondo specifiche industriali [13, 10, 14, 17]. BeamNG, al contrario, non integra nativamente questo

formato, il che rende necessario l'uso di strumenti intermedi, come il convertitore sviluppato in questo lavoro di tesi, per garantire interoperabilità [12, 18]. Un ulteriore elemento di confronto riguarda le **interfacce API Python** [12, 7]. Entrambi i simulatori consentono un controllo programmato tramite linguaggio Python, ma con differenze significative: CARLA offre una libreria potente ma complessa da utilizzare, soprattutto per la gestione simultanea di molteplici agenti [17, 8]. BeamNG, invece, sfrutta il modulo `beamngpy`, più leggero e intuitivo, che semplifica la comunicazione con il simulatore e l'estrazione dei dati [18, 12, 9]. Il **supporto per macOS** rappresenta una limitazione pratica per CARLA, che non prevede una versione nativa e richiede workaround tramite Docker, spesso non ottimali [17]. BeamNG, pur non garantendo un supporto completo, può essere utilizzato su macOS tramite configurazioni dedicate e virtualizzazione, risultando quindi relativamente più accessibile [18, 12]. Infine, per quanto riguarda la **stabilità**, CARLA si dimostra generalmente solido ma presenta alcune criticità in scenari complessi o con un elevato numero di attori, dove possono emergere problemi di sincronizzazione [17, 7]. BeamNG, invece, si distingue per una maggiore stabilità e continuità di esecuzione, aspetto che lo rende particolarmente adatto a esperimenti di lunga durata e raccolta dati estensiva [18, 9]. In conclusione, i due simulatori si presentano come complementari: CARLA è più adatto alla simulazione di scenari di traffico realistici e all'integrazione con agenti intelligenti, mentre BeamNG si rivela superiore nella precisione fisica e nell'analisi delle collisioni [12, 3, 4]. L'utilizzo combinato di entrambi, come dimostrato in questo lavoro, consente di ottenere un framework di test più completo e bilanciato [5, 7, 6]. Entrambi i simulatori offrono strumenti validi e potenti per la simulazione di scenari di guida autonoma, ma con caratteristiche complementari [12, 3]. CARLA si presta maggiormente a test in ambienti controllati con definizione formale (XOSC), mentre BeamNG eccelle nella simulazione fisica realistica e nel controllo dinamico degli attori [8, 18, 9]. Il nostro lavoro ha saputo integrare le potenzialità di entrambi, garantendo flessibilità, realismo e un'adequata varietà di scenari [5, 7, 6, 4].

3.1 Struttura del progetto

Il progetto si concentra sull’ottimizzazione della selezione degli scenari di test per sistemi di assistenza alla guida (ADAS, Advanced Driver Assistance Systems), sfruttando due simulatori complementari: **CARLA** [17] e **BeamNG.tech** [18]. CARLA, basato su Unreal Engine, abilita la definizione di scenari urbani complessi e l’interazione con agenti intelligenti; BeamNG.tech, fondato su un motore di fisica soft-body, consente di modellare dinamiche di collisione e deformazioni veicolari con elevato realismo. L’obiettivo principale è duplice: (i) identificare scenari *critici* (collisioni, sorpassi azzardati, attraversamenti con semaforo rosso) e (ii) garantire la *diversità* degli scenari selezionati, così da massimizzare la copertura del dominio operativo minimizzando al contempo i tempi di esecuzione. Questo obiettivo si colloca nel solco delle più recenti ricerche sul testing basato su simulazione per veicoli autonomi, che mostrano come una quota non trascurabile di test generati automaticamente sia poco informativa e produca sprechi computazionali [1, 6]. Per conseguire tali finalità, la metodologia è articolata in due macro-fasi. Per comprendere a pieno le due fasi successive abbiamo bisogno di una fase preliminare che ci permetta di avere un excursus generale del progetto.

Fase preliminare

La fase preliminare affronta una criticità strutturale: l'assenza di scenari JSON nativamente compatibili con BeamNG.tech. È stata quindi sviluppata un'infrastruttura di conversione capace di trasformare scenari descritti per CARLA (formato .xosc) in rappresentazioni JSON eseguibili in BeamNG.tech, preservando la semantica degli elementi rilevanti per la simulazione. L'obiettivo non è stata la copertura integrale dello standard OpenSCENARIO, bensì la definizione di una **pipeline minima ma robusta** per trasferire gli elementi essenziali: attori, posizioni e traiettorie chiave, condizioni ambientali e regole di attivazione degli eventi. Questa scelta ha consentito di garantire interoperabilità tra i due simulatori senza dover gestire costrutti raramente utilizzati dello standard. L'architettura del convertitore si articola in quattro stadi: **Parse** (lettura e navigazione della struttura XML del file .xosc), **Normalize** (risoluzione dei riferimenti parametrici e normalizzazione delle unità di misura), **Map** (traduzione degli elementi OpenSCENARIO in costrutti BeamNG.tech) e **Emit** (generazione del documento JSON finale). Gli elementi principali gestiti includono attori e veicoli ego, condizioni iniziali di posizionamento, parametri meteorologici e temporali, comportamenti dinamici quali variazioni di velocità e cambi di corsia, nonché trigger di avvio e arresto basati su condizioni temporali o spaziali. Questa infrastruttura ha reso possibile l'esecuzione comparativa degli scenari su entrambi i simulatori, costituendo la base per la successiva analisi delle metriche di interesse.

Prima fase

riguarda la valutazione di scenari descritti in formati standardizzati — OpenSCENARIO (.xosc) per CARLA e JSON per BeamNG.tech —, così da consentire sia un confronto sistematico tra simulatori, sia l'allineamento a prassi industriali. I dati grezzi delle simulazioni sono elaborati da strumenti di parsing e analisi (ad es., `run_and_log_scenarios.py`, `carla_to_beamng.py`) per l'estrazione e l'aggregazione delle metriche sopra citate. Nel complesso, l'infrastruttura integra (i) esecuzione automatizzata degli scenari, (ii) registrazione strutturata degli eventi e (iii) analisi delle metriche derivate, costituendo la base per una selezione *costo-efficace* degli scenari. Tale approccio è coerente con le proposte recenti (ad es. *SDC-Scissor*)

che impiegano modelli di Machine Learning per predire a priori la rilevanza degli scenari, riducendo l'esecuzione di test ridondanti e migliorando l'efficienza del processo di validazione [1, 6].

Seconda fase

si valutano scenari dinamici in **CARLA** mediante script Python proprietari (ad es., `ego_traffic.py`, `loop_runner.py`), che orchestrano il traffico, acquisiscono telemetria e registrano eventi in formato JSON. I tracciati contengono, tra le altre, la tipologia di evento (collisione / assenza di incidenti), le caratteristiche della mappa (semafori, curve, incroci), la tipologia di strada, nonché le condizioni meteorologiche al momento dell'evento. Tali dati alimentano il calcolo delle metriche di interesse: (i) *tempo di esecuzione*, (ii) *criticità* e (iii) *diversità*.

Scelte di progettazione. Il design del convertitore è stato guidato da alcune scelte fondamentali. In primo luogo, è stato adottato un **subset pragmatico dello standard**, privilegiando le sezioni OpenSCENARIO più diffuse e direttamente traducibili; elementi complessi, come i **TrafficSignalController** avanzati, sono supportati in forma semplificata oppure marcati come **unsupported** con meccanismi di fallback sicuri. In secondo luogo, i **waypoint sono stati scelti come lingua franca**: traiettorie e manovre vengono ridotte a sequenze di waypoint con vincoli di velocità o precedenza, così da garantire un'esecuzione stabile nel motore fisico di BeamNG. Inoltre, è stata applicata la **normalizzazione delle unità**, con tutte le grandezze espresse in unità SI (metri, secondi, m/s), al fine di evitare ambiguità nella fase di esecuzione. Infine, si è mantenuta la **coerenza semantica**, traducendo le azioni con dinamica di tipo **step** o **linear** come cambi di set-point a tratti (segmenti di traiettoria) o come vincoli temporizzati all'interno del blocco AI.

Schema JSON di output. Di seguito un esempio semplificato di documento JSON generato dal convertitore `.xosc` $\rightarrow$ BeamNG:

```
{
  "scenario_id": "FollowLeadingVehicle_xosc_Town01",
  "map": "Town01",
```

```
"weather": {
  "cloudiness": 70.0,
  "precipitation": 20.0,
  "fog": 0.0,
  "sun_altitude_deg": 45.0,
  "time_of_day": "2020-03-20T12:00:00"
},
"actors": [
  {
    "name": "hero",
    "type_id": "vehicle.lincoln.mkz_2017",
    "role": "ego",
    "spawn": { "x": 190.0, "y": 133.0, "z": 0.0, "yaw": 0.0 }
  },
  {
    "name": "adversary",
    "type_id": "vehicle.tesla.model3",
    "role": "traffic",
    "spawn_relative_to": "hero",
    "offset": { "ds": 50.0, "dt": -1.0 }
  }
],
"routes": [
  {
    "actor": "adversary",
    "waypoints": [
      { "x": 195.0, "y": 133.0, "speed": 2.0 },
      { "x": 300.0, "y": 140.0, "speed": 2.0 }
    ]
  }
],
"ai": {
  "behavior": "keep_lane",
  "max_speed": 8.3,
```

```
    "allow_red_light": false,  
    "allow_speeding": false  
  },  
  "termination": {  
    "timeout_s": 60,  
    "min_travelled_m": 150  
  }  
}
```

Validazione ed error handling. Prima dell’emissione, il documento è sottoposto a controlli di coerenza (presenza di mappa, almeno un attore, *spawn* validi, *waypoint* con coordinate finite, vincoli temporali sensati). Gli elementi non mappabili sono annotati e disattivati nel JSON attraverso flag espliciti, mantenendo la tracciabilità delle parti scartate. In caso di riferimenti parametrizzati non risolvibili, il tool applica *default* conservativi (velocità e timeout minimi) e segnala l’evento nel log di conversione.

Limitazioni attuali e possibili estensioni. Il convertitore copre i casi d’uso più ricorrenti (veicoli, *teleport*, *speed actions*, *start/stop triggers* semplici, meteo), mentre tratta in modo parziale pedoni complessi, reti semaforiche articolate e interazioni multi-agente basate su regole avanzate. Tra le evoluzioni pianificate rientrano: l’ampliamento del supporto a *TrafficSignalController*, la traduzione diretta di *LaneChangeAction* parametriche in *waypoint* laterali generati proceduralmente, e l’introduzione di profili AI differenziati (roaming, aggressive, cautious) selezionabili da JSON.

Integrazione nella pipeline. Il convertitore è progettato per operare come passaggio opzionale: quando lo scenario d’ingresso è *.xosc*, il tool produce il JSON compatibile con BeamNG che viene poi eseguito dal *runner* di simulazione; quando invece si utilizzano scenari Python per CARLA, la fase di parsing/conversione non è necessaria. In entrambi i casi, i risultati di esecuzione confluiscono nella medesima struttura di output (file JSON di *run* contenenti tempo alla collisione, meteo al momento dell’evento, caratteristiche statiche della mappa e stato finale dello scenario),

così da alimentare in modo omogeneo il modulo di selezione/ottimizzazione degli scenari.

3.2 Fase preliminare

3.2.1 Generazione e parsing degli scenari

Uno dei principali ostacoli iniziali è stata l'assenza di scenari *JSON* già pronti per *BeamNG*. Per colmare tale gap è stato sviluppato un **tool di conversione** dedicato che, dato un file `.xosc` preparato per *CARLA*, ne estrae le informazioni essenziali e le traduce in un **formato JSON** eseguibile in *BeamNG*. L'obiettivo progettuale non è stato la copertura integrale dello standard, bensì la definizione di una *pipeline minima ma robusta* capace di trasferire gli elementi realmente necessari alla simulazione: attori, posizioni e traiettorie chiave, condizioni ambientali e regole di attivazione più comuni.

Architettura del convertitore. Il convertitore segue una pipeline in quattro stadi:

1. **Parse** (*lxml.etree*): lettura del file `.xosc` e navigazione della struttura XML per accedere a `Entities`, `Storyboard`, `Init`, `ManeuverGroup`, `Event/Action` e `Trigger`.
2. **Normalize**: risoluzione di `ParameterDeclarations` e sostituzione dei riferimenti parametrizzati (ad es. velocità target o offset), normalizzazione delle unità (tempi in secondi, distanze in metri, angoli in gradi) e consolidamento dei riferimenti a strade/corsie in un insieme di *waypoint* operativi.
3. **Map**: traduzione degli elementi *OpenSCENARIO* in costrutti *BeamNG*: *spawn points* per gli attori, liste di *waypoint* come traiettorie, *AI modes* e vincoli cinematici (ad es. vincoli di velocità), meteo e ora del giorno.
4. **Emit**: generazione del documento JSON finale, conforme a uno schema leggero adottato nel progetto (chiavi ben definite, valori controllati, campi opzionali con *fallback* sensati).

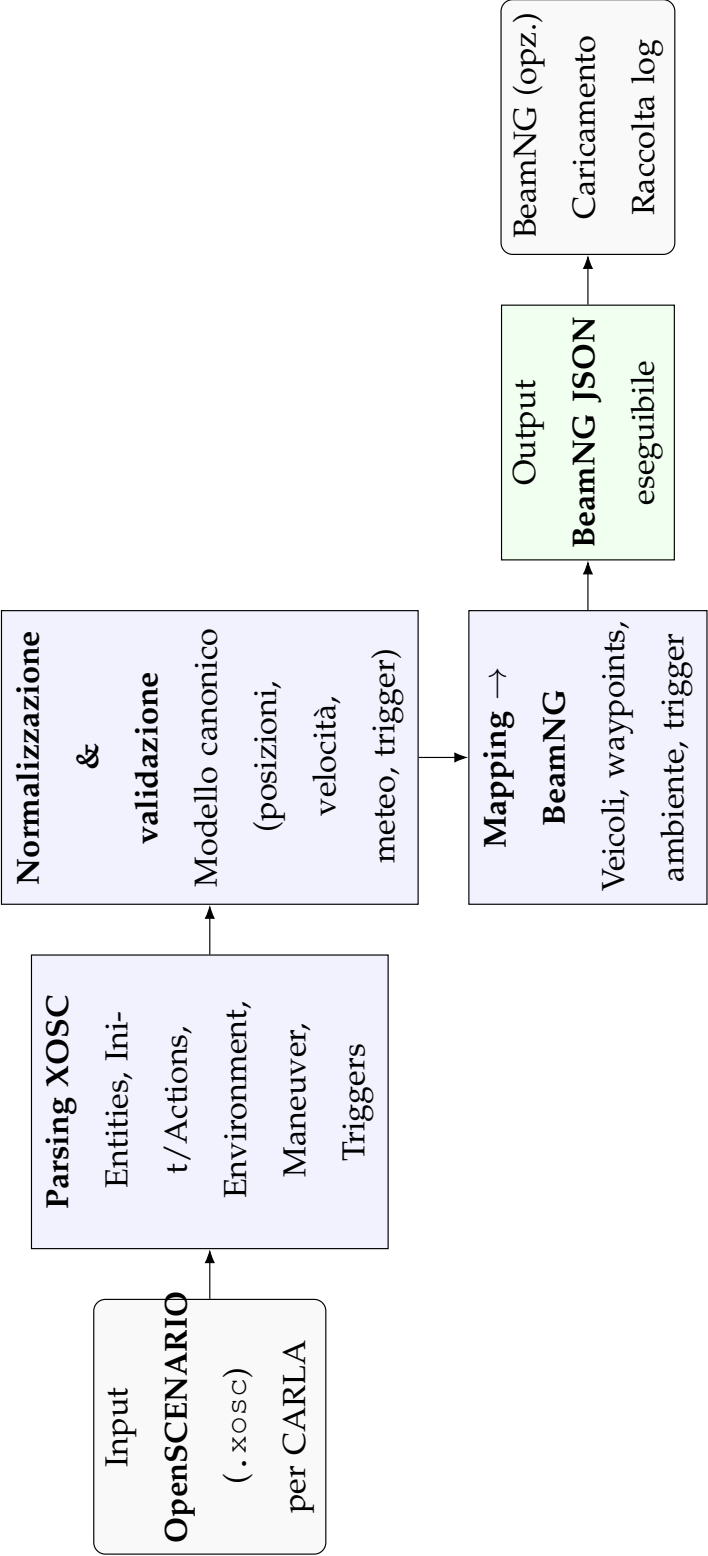


Figura 3.1: Pipeline del convertitore da OpenSCENARIO (.xosc) per CARLA a JSON eseguibile in BeamNG: parsing, normalizzazione, mapping e generazione dell’output.

3.2.2 Mappatura degli elementi OpenSCENARIO nel convertitore

Gli elementi principali gestiti dal convertitore includono diverse categorie fondamentali. La struttura adottata segue le specifiche dello standard ASAM OpenSCENARIO [13], un formato ampiamente utilizzato per la descrizione di scenari dinamici nel testing di sistemi di guida autonoma, che consente di rappresentare in modo standardizzato attori, traiettorie, condizioni ambientali e comportamenti temporali [1]. Anzitutto, gli **attori**, ad esempio quelli definiti nella sezione *Entities/ScenarioObject*, rappresentano i veicoli con il loro identificativo di tipo, le dimensioni e gli attributi essenziali; tra questi, ...il veicolo ego viene etichettato esplicitamente per consentire la corretta orchestrazione della simulazione. La distinzione tra veicolo ego e attori non-player character (NPC) è fondamentale nei framework di test selection basati su machine learning [1], poiché le interazioni ego-NPC costituiscono la base per l'estrazione di feature comportamentali utilizzate nella predizione della rilevanza degli scenari. Le **condizioni iniziali**, come quelle contenute in *Init/Actions* e descritte tramite istruzioni quali *TeleportAction* o *Position* (nelle varianti *RoadPosition*, *LanePosition* e *RelativeRoadPosition*), vengono convertite in posizioni e direzioni di partenza precise, preservando eventuali offset e orientamenti quando specificati. Le informazioni relative a **meteo e ora del giorno**, corrispondenti a *EnvironmentAction* (nuvolosità, pioggia, nebbia, intensità del sole) o *TimeOfDay*, sono normalizzate e riportate in un blocco coerente con lo schema JSON adottato. Anche i **comportamenti dinamici**, come quelli descritti in *Maneuver/Event/Action*, ad esempio le *SpeedAction* con *AbsoluteTargetSpeed*, sono tradotti in vincoli o target di velocità da rispettare lungo i segmenti di traiettoria, mentre eventuali *LaneChangeAction* o offset laterali vengono resi come waypoint a spostamento controllato. Infine, i **trigger di avvio e arresto**, ovvero le sezioni *StartTrigger/StopTrigger* basate su condizioni temporali o spaziali (come *SimulationTimeCondition*, *TraveledDistanceCondition* o *RelativeDistanceCondition*), sono mappati in criteri di avvio e terminazione che stabiliscono, ad esempio, un timeout massimo, una distanza minima percorsa o il completamento della traiettoria.

3.3 Prima fase

La metodologia proposta per l’ottimizzazione del processo di validazione dei sistemi ADAS si articola attraverso un’architettura computazionale che integra l’elaborazione automatizzata di scenari standardizzati con tecniche avanzate di selezione basate su modelli predittivi. L’adozione di formati interoperabili, quali OpenSCENARIO (.xosc) per CARLA e JSON per BeamNG.tech, costituisce il fondamento tecnologico per garantire la riproducibilità e la comparabilità dei risultati sperimentali, conformemente agli standard industriali emergenti nel dominio automotive. L’infrastruttura sviluppata implementa una pipeline computazionale che orchestra l’esecuzione parallela degli scenari attraverso componenti software specializzati (`run_and_log_scenarios.py` `carla_to_beamng.py`), permettendo l’acquisizione sistematica di telemetria ad alta risoluzione e la successiva estrazione di metriche quantitative. Il framework così concepito si allinea con il paradigma metodologico proposto da *SDC-Scissor* [1], che dimostra come l’applicazione di tecniche di apprendimento automatico supervisionato possa ridurre significativamente lo spazio di ricerca degli scenari critici attraverso la predizione della loro rilevanza prima dell’esecuzione effettiva. SDC-Scissor introduce infatti un modello di classificazione binaria basato su Random Forest che, analizzando le caratteristiche statiche degli scenari, determina con accuratezza superiore all’85% quali test produrranno violazioni delle proprietà di sicurezza, consentendo una riduzione del carico computazionale fino al 70% rispetto all’esecuzione esaustiva [6]. Uno degli aspetti metodologicamente più rilevanti del progetto riguarda la gestione degli scenari di test. La scelta di concentrarsi sui formati `xosc` e `json` non è casuale: il primo rappresenta lo standard aperto *OpenSCENARIO*, largamente adottato in ambito accademico e industriale per la descrizione di scenari di guida, mentre il secondo offre un formato leggero, flessibile e facilmente integrabile in pipeline di analisi e strumenti di machine learning. Basare l’intera infrastruttura su questi due formati ha permesso di garantire al tempo stesso aderenza agli standard e efficienza nell’elaborazione dei dati. Per rendere questo processo scalabile è stato sviluppato uno *scenario runner* apposito. La sua progettazione risponde a due esigenze metodologiche fondamentali. Da un lato, ridurre l’intervento manuale, evitando errori umani e permettendo l’elaborazione

automatizzata di un numero elevato di scenari in sequenza; dall'altro, garantire la **comparabilità** dei risultati imponendo regole uniformi di esecuzione (avvio, gestione del traffico, condizioni di arresto) e registrazione dei dati. L'alternativa, ossia una selezione manuale o semi-automatica degli scenari, avrebbe portato a dataset eterogenei e difficilmente utilizzabili per un'analisi quantitativa, compromettendo la validità statistica dei risultati. Il *runner* esegue dunque ogni scenario all'interno del simulatore CARLA e raccoglie in maniera sistematica tre metriche chiave: il **tempo di esecuzione**, utile a stimare il costo computazionale e operativo di uno scenario; l'**indice di criticità**, che deriva dal verificarsi di eventi potenzialmente pericolosi come collisioni, frenate improvvise o condizioni meteo avverse; e il **punteggio di diversità**, che misura quanto uno scenario differisca dagli altri già analizzati, così da evitare ridondanze e coprire un ventaglio più ampio di situazioni. La scelta di queste tre metriche è motivata dal loro legame diretto con le sfide individuate in letteratura: riduzione degli scenari "uninformativi" che consumano risorse senza apportare conoscenza.

3.3.1 Script sviluppati per la prima parte

La prima parte del progetto è stata supportata dallo sviluppo di una serie di script pensati non solo come strumenti operativi, ma come componenti metodologici di una pipeline di test automatizzata. Lo script `run_and_log_scenarios.py` rappresenta il cuore del processo: la sua funzione non si limita a lanciare scenari in sequenza, ma consiste nel definire una regola uniforme di esecuzione (avvio, monitoraggio degli eventi, condizione di arresto per collisione o timeout) e un formato strutturato per il salvataggio dei risultati. Questa scelta metodologica nasce dall'esigenza di ridurre la variabilità dovuta a esecuzioni manuali, garantendo che ogni scenario venga trattato secondo procedure identiche e produca un output omogeneo. Un'alternativa possibile sarebbe stata quella di registrare soltanto eventi salienti (ad esempio, la presenza o meno di collisioni), ma ciò avrebbe sacrificato la granularità del dato, limitando le successive analisi. La scelta di registrare in modo esteso (tempo esatto, entità coinvolte, coordinate, meteo, tipologia stradale) consente invece di alimentare processi più sofisticati, come il calcolo della criticità o la classificazione delle tipologie

di incidente. Accanto a questo modulo, lo script `diversity_extractor.py` si occupa del calcolo del punteggio di diversità. La motivazione alla base di questa scelta è duplice. Da un lato, in letteratura è ampiamente riconosciuto che una delle criticità del testing in simulazione è l'elevata ridondanza: scenari formalmente diversi finiscono spesso per esercitare comportamenti quasi identici nel veicolo ego. Dall'altro lato, la generazione degli scenari tramite script Python introduce una componente di casualità: replicare lo stesso scenario più volte non avrebbe senso, poiché non porterebbe nuove informazioni. In questo contesto, una misura di diversità diventa essenziale per selezionare insiemi di test che non si sovrappongano e che massimizzino la copertura delle possibili condizioni di guida. Senza tale misura, si rischierebbe di dedicare tempo computazionale a eseguire scenari che, pur avendo parametri di input diversi, producono effetti indistinguibili.

3.3.2 Raccolta dei risultati per la prima parte

Tutti i risultati vengono raccolti sotto forma di file `.json`, un formato scelto non per convenzione, ma per precise ragioni metodologiche. Rispetto a formati più rigidi (ad esempio CSV o XML), il JSON garantisce una rappresentazione flessibile e annidata, capace di includere sia valori numerici (tempi, punteggi) sia strutture complesse (descrizione del meteo, caratteristiche della mappa, coordinate tridimensionali). Questa scelta permette di preservare un alto livello di dettaglio senza sacrificare la leggibilità e l'interoperabilità con strumenti di analisi (pandas, NumPy, librerie ML).

L'organizzazione dei risultati segue tre direttrici fondamentali:

- `execution_time.json`: documenta i tempi di esecuzione e costituisce una misura del “costo” computazionale di ogni scenario;
- `criticality_score.json`: quantifica il livello di rischio sulla base degli eventi osservati;
- `diversity_score.json`: rappresenta la distanza semantica tra scenari, evitando sovrapposizioni informative.

La decisione di separare le tre metriche in file distinti risponde a un principio di modularità: consente infatti di interrogare direttamente un singolo aspetto (ad

esempio solo la criticità) oppure di integrare i dati in modo incrociato per processi di selezione più complessi. In sintesi, questa architettura di logging e raccolta dati è stata concepita per rispondere a tre sfide centrali: la non ripetibilità degli scenari (che rende necessario estrarre il massimo da ogni singola esecuzione), la ridondanza potenziale (affrontata tramite la diversità), e la necessità di dati ricchi e strutturati per alimentare metodi di selezione e ottimizzazione.

3.4 BehaviorAgent e gli ADAS

Prima di procedere con la descrizione della seconda fase, è necessario chiarire un aspetto metodologico fondamentale riguardante il ruolo del BehaviorAgent utilizzato nelle simulazioni CARLA. Il BehaviorAgent **non rappresenta un sistema ADAS** né tantomeno un modulo di guida autonoma da testare: si tratta invece di un **agente di controllo** fornito nativamente dalla piattaforma CARLA per simulare comportamenti di guida realistici e ripetibili. La sua funzione, nel contesto di questo progetto, è quella di *pilotare* il veicolo ego secondo logiche di navigazione predefinite (seguimento di traiettorie, rispetto dei limiti di velocità, gestione delle precedenza), così da creare le condizioni operative nelle quali gli scenari critici possono emergere naturalmente dall'interazione con il traffico circostante, i pedoni e le condizioni ambientali. La distinzione è cruciale dal punto di vista metodologico: l'oggetto della valutazione non è la qualità decisionale del BehaviorAgent (che è dato per assunto come baseline funzionante), bensì la **capacità degli scenari generati di sollecitare situazioni critiche** — collisioni, violazioni semaforiche, manovre improvvise — che, in un contesto applicativo reale, metterebbero alla prova i sistemi ADAS o di guida autonoma integrati nel veicolo. In altre parole, il BehaviorAgent funge da *surrogato operativo* di un guidatore umano o di un sistema di controllo da validare, permettendo di concentrare l'attenzione sulla **caratterizzazione e selezione degli scenari** piuttosto che sull'implementazione di logiche di guida proprietarie. Questa scelta progettuale offre diversi vantaggi. In primo luogo, garantisce **riproducibilità e confrontabilità** tra esecuzioni diverse, eliminando la variabilità introdotta da controllori custom non standardizzati. In secondo luogo, riduce significativamente i tempi di sviluppo e validazione, consentendo di concentrare le risorse sul problema

centrale: la generazione, esecuzione e selezione di scenari informativi. Infine, rende il framework facilmente estensibile: qualora si volesse integrare un sistema ADAS reale o un controller proprietario, sarebbe sufficiente sostituire il `BehaviorAgent` mantenendo inalterata l'intera pipeline di logging, analisi e selezione degli scenari. In sintesi, il `BehaviorAgent` è uno **strumento metodologico**, non l'oggetto di studio: la sua presenza consente di costruire un ambiente di test controllato e realistico nel quale valutare la capacità degli scenari di elicitare comportamenti critici, indipendentemente dall'implementazione specifica del sistema di guida che, in una fase successiva o in un contesto industriale, prenderebbe il suo posto.

3.5 Seconda fase

3.5.1 Gestione delle esecuzioni

La seconda parte della metodologia nasce dall'esigenza di osservare fenomeni emergenti che gli scenari dichiarativi in `.xosc` faticano a catturare. Per questo motivo è stato sviluppato lo script `ego_traffic.py`, concepito per orchestrare simulazioni dinamiche in CARLA con un veicolo *ego* inserito in contesti urbani ed extraurbani variabili, popolati da traffico autonomo, pedoni e segnali semaforici, sotto condizioni meteorologiche stocastiche. La scelta di utilizzare il `BehaviorAgent` come controllore primario dell'*ego* non è meramente implementativa: consente di imporre un comportamento di guida ragionevole e conforme alle regole, mantenendo tuttavia la possibilità di iniettare variabilità controllata — ad esempio brusche decelerazioni, sorpassi o attraversamenti pedonali non prevedibili — così da sollecitare i sistemi ADAS in situazioni plausibili ma non rigidamente pre-programmate. Un'alternativa basata su un controllore proprietario avrebbe richiesto un costo di modellazione e validazione non compatibile con l'obiettivo di eseguire molteplici run eterogenee in tempi contenuti; il `BehaviorAgent`, invece, offre un compromesso efficace tra realismo, riproducibilità del profilo di guida e rapidità di sviluppo. Un ulteriore elemento metodologico è la regola di arresto uniforme basata sulla prima collisione o, in mancanza, su un timeout di durata prefissata: tale scelta privilegia l'informazione *time-to-failure*, rende confrontabili esecuzioni differenti ed evita che la dinamica post-

incidente contaminino le misure di contesto. Per garantire indipendenza tra scenari consecutivi, la mappa viene ricaricata a ogni run, così da prevenire effetti di stato residuo. Si è deliberatamente rinunciato a repliche dello stesso scenario con identici semi casuali: poiché la generazione è intenzionalmente stocastica, ripetere la medesima configurazione avrebbe un valore informativo marginale rispetto all'esplorazione di nuovi campioni dello spazio scenario. L'obiettivo, coerente con la letteratura sulla *test selection* in simulazione, è massimizzare la varietà di condizioni osservate a parità di budget computazionale, riducendo l'esecuzione di test ridondanti o scarsamente informativi.

3.5.2 Raccolta dei dati nella seconda parte

La raccolta dei dati è stata progettata per essere *event-driven*, coerente con la finalità di selezione: ogni esecuzione produce un singolo artefatto informativo che coglie il momento e il contesto dell'evento critico (o del timeout) e ne arricchisce la semantica con attributi ambientali e topologici. La serializzazione in `.json` è stata preferita ad alternative come CSV o database relazionali perché permette di conservare strutture annidate (ad esempio dettagli meteo, caratteristiche della mappa e coordinate spaziali) senza perdita di informazione, favorendo al contempo portabilità e integrazione diretta con strumenti di analisi (p. es. `pandas` e pipeline di machine learning). La decisione di mantenere un file *atomico* per l'esecuzione semplifica il tracciamento degli errori e isola gli scenari, condizione essenziale quando le esecuzioni sono indipendenti e non ripetute artificialmente. L'architettura metrica implementata riconosce la natura intrinsecamente stocastica dell'ambiente simulativo, dove le dinamiche emergenti dal traffico autonomo, le condizioni meteorologiche variabili e i comportamenti probabilistici degli agenti introducono elementi di variabilità non deterministica. Il tempo di esecuzione, registrato fino all'evento terminale o al raggiungimento del timeout, costituisce una misura diretta del carico computazionale richiesto da ciascuno scenario, sebbene sia soggetto a fluttuazioni significative derivanti dall'interazione complessa tra il veicolo ego e l'ambiente dinamico circostante. I dati empirici raccolti evidenziano infatti un'ampia dispersione temporale, con valori che spaziano da esecuzioni rapide inferiori al secondo fino a simulazioni che saturano

il timeout predefinito, riflettendo la diversità intrinseca delle situazioni di guida generate. L'indicatore di criticità adotta un paradigma *event-driven* che quantifica il rischio attraverso l'occorrenza di eventi discreti quali collisioni, violazioni semaforiche e manovre di emergenza, garantendo una caratterizzazione semanticamente stabile e incrementabile. Tale approccio supera le limitazioni degli indici continui basati su dinamiche relative, che richiederebbero la calibrazione di numerosi iperparametri e introdurrebbero dipendenze da modelli di rischio non ancora consolidati nel dominio automotive. La caratterizzazione contestuale degli eventi, che incorpora la topologia stradale (presenza di curve, incroci, semafori), le condizioni meteorologiche istantanee e la tipologia del tratto percorso, arricchisce la dimensionalità informativa di ciascun episodio simulativo, consentendo analisi di correlazione tra fattori ambientali e criticità emergente. La metrica di diversità, infine, affronta la problematica della ridondanza informativa attraverso la quantificazione della dissimilarità semantica tra scenari, garantendo che il sottoinsieme selezionato per la validazione massimizzi la copertura dello spazio operativo. Questa misura si rivela particolarmente cruciale nel contesto di scenari generati proceduralmente, dove la componente stocastica potrebbe produrre variazioni parametriche che, pur essendo formalmente distinte, risultano funzionalmente equivalenti dal punto di vista del testing. In questa fase, un disegno sperimentale snello, basato su logging semantico essenziale ma completo, ha garantito affidabilità delle misure, semplicità di analisi e un favorevole rapporto tra informazione estratta e tempo di calcolo. In sintesi, la combinazione di controller realistico, variabilità stocastica controllata, regola d'arresto univoca e serializzazione *evento-centrica* ha dimostrato di funzionare bene perché allinea la raccolta dati con l'obiettivo finale di selezionare scenari critici e non ridondanti, massimizzando la copertura del dominio operativo a parità di budget.

4.1 Architettura del Sistema

Il sistema ASAT (ADAS Scenario Analysis Tool) è stato progettato seguendo il paradigma dell'architettura a microservizi, organizzato in quattro componenti principali:

- **Modulo Parser:** responsabile della conversione tra formati (.xosc → JSON)
- **Modulo di Esecuzione:** gestisce l'orchestrazione delle simulazioni
- **Modulo di Feature Extraction:** estrae metriche quantitative dagli scenari
- **Modulo di Analisi:** calcola criticality, diversity e execution time

L'architettura adotta il pattern Observer per il monitoraggio degli eventi e Strategy per l'implementazione multi-simulatore.

4.1.1 La creazione del parser

All'interno del tool sviluppato è stata implementata una componente di parsing in grado di effettuare la conversione automatizzata degli scenari definiti in

formato OpenSCENARIO (`.xosc`), utilizzato da CARLA, verso il formato JSON compatibile con BeamNG. Il parser è stato progettato e sviluppato interamente in linguaggio Python, seguendo un'architettura modulare che garantisce efficienza computazionale e perfetta integrazione nel framework complessivo. La scelta di Python come linguaggio di implementazione deriva dalla necessità di mantenere coerenza tecnologica con l'ecosistema dei simulatori e di facilitare l'estensibilità futura del componente. L'architettura del parser si articola attraverso una pipeline di trasformazione multi-stadio che preserva la semantica degli scenari durante il processo di conversione. Il modulo di parsing analizza la struttura gerarchica XML degli scenari OpenSCENARIO, estraendo e normalizzando gli elementi fondamentali quali traiettorie veicolari, condizioni iniziali, trigger eventi e parametri ambientali. Successivamente, un componente di mappatura semantica traduce questi elementi nel modello di rappresentazione di BeamNG, gestendo le differenze strutturali tra i due formati e applicando le necessarie trasformazioni coordinate e temporali. L'integrazione di questo modulo consente ora di generare scenari per BeamNG direttamente a partire da quelli già esistenti per CARLA, preservando le informazioni strutturali e cinematiche originarie. In questo modo, l'utente finale non solo può eseguire l'estrazione delle caratteristiche, il calcolo della probabilità di fallimento e le analisi critiche descritte in precedenza, ma ha anche la possibilità di creare un dataset BeamNG ex novo, sfruttando al massimo le risorse già in suo possesso e ampliando notevolmente le possibilità di testing e ottimizzazione.

4.1.2 La struttura del parser

Il parser, implementato nel modulo `carlatobeamng.py`, si articola in tre fasi principali:

Parsing del file `.xosc` La fase iniziale prevede il caricamento e l'analisi del file `.xosc` mediante la libreria `lxml.etree`, che permette un parsing efficiente dei file XML. Durante questa fase, il parser estrae informazioni chiave dallo scenario, tra cui:

- Condizioni meteorologiche: vengono recuperati parametri relativi allo stato delle nuvole, l'intensità del sole, la presenza di nebbia e le precipitazioni. Questi

dati sono strutturati in un sotto blocco denominato "weather script".

- Segnaletica stradale: nel caso siano presenti controller di segnali di traffico (TrafficSignalController), viene generato un blocco "traffic script" che riporta il limite di velocità (inizialmente fittizio) e l'identificativo del controller.
- Waypoints: il parser individua i punti di controllo della traiettoria (LanePosition) e li organizza in una lista di waypoint, ognuno caratterizzato dalle coordinate spaziali lungo la strada.

Architettura della Pipeline di Conversione

La pipeline di conversione da OpenSCENARIO a BeamNG.tech è organizzata secondo un'architettura a cinque stadi sequenziali, ciascuno responsabile di una specifica trasformazione dei dati.

Stadio 1 - Parsing e Validazione. Il file XOSC viene inizialmente processato da un parser XML che costruisce una rappresentazione DOM (Document Object Model) della struttura. Questa rappresentazione viene successivamente validata contro lo schema XSD di OpenSCENARIO versione 1.2 per garantire la conformità sintattica e semantica.

Stadio 2 - Estrazione Semantica. Dal DOM validato vengono estratti gli elementi semantici dello scenario attraverso query XPath mirate. Gli elementi estratti includono: (i) definizioni delle entità veicolari con relative proprietà dinamiche, (ii) topologia della rete stradale e waypoint associati, (iii) configurazione ambientale comprensiva di condizioni meteorologiche e illuminazione, (iv) sequenza di azioni e trigger definiti nello storyboard.

Stadio 3 - Trasformazione del Modello. Gli elementi estratti subiscono una trasformazione per adattarsi al modello di dati di BeamNG.tech. Questa fase include la mappatura dei tipi di veicolo XOSC verso i modelli disponibili in BeamNG, la conversione del sistema di riferimento spaziale da ISO 8855 al sistema di coordinate di BeamNG, e l'adattamento dei parametri fisici alle unità di misura del simulatore target.

Stadio 4 - Sintesi delle Azioni. Le azioni definite nello storyboard OpenSCENARIO vengono tradotte in comandi eseguibili da BeamNG. Le traiettorie continue vengono discretizzate attraverso interpolazione cubica per generare sequenze di way-

point, mentre le azioni istantanee vengono mappate sui corrispondenti trigger del simulatore.

Stadio 5 - Serializzazione. La struttura dati risultante viene serializzata in formato JSON seguendo lo schema richiesto da BeamNG.tech, con validazione finale della completezza e coerenza della configurazione generata.

4.2 Seconda parte: simulazioni e script Python

4.2.1 Simulazioni automatiche, gestione del traffico e raccolta dei risultati

La fase centrale del progetto è costituita dallo sviluppo di un'infrastruttura automatizzata in grado di gestire, eseguire e monitorare un vasto insieme di scenari simulativi in CARLA, con l'obiettivo di raccogliere dati utili per alimentare successivamente un algoritmo di selezione ottimizzata. Per realizzare questa pipeline sono stati sviluppati script Python dedicati, in particolare il file `ego_traffic.py`, che rappresenta il modulo principale per l'esecuzione e il monitoraggio delle simulazioni.

Struttura generale dello script Lo script `ego_traffic.py` è stato concepito per lanciare scenari in modo completamente automatico, eseguendoli uno dopo l'altro senza necessità di intervento umano. Ogni scenario prevede l'inizializzazione di:

- Un **veicolo ego** controllato direttamente tramite il modulo `BehaviorAgent` incluso nella libreria `carla.agents.navigation`. L'agente è responsabile della navigazione autonoma del veicolo seguendo una destinazione predefinita e interagendo attivamente con l'ambiente simulato.
- Una rete dinamica di **veicoli NPC** e **pedoni**, generati con caratteristiche probabilistiche per garantire variabilità comportamentale da uno scenario all'altro.

Scelta delle mappe e condizioni ambientali Ad ogni esecuzione viene selezionata casualmente una mappa tra quelle disponibili in CARLA (`Town01–Town07`), ciascuna con caratteristiche stradali differenti (numero di incroci, curve, semafori, ecc.). Le

condizioni meteo vengono impostate all’inizio in modo casuale e aggiornate dinamicamente ogni 10 secondi, tra diverse configurazioni preimpostate (sole, pioggia, nebbia, nuvole, bassa visibilità, ecc.).

Logica comportamentale avanzata del traffico Per aumentare il grado di realismo e generare situazioni potenzialmente critiche per i sistemi ADAS, lo script implementa una serie di comportamenti probabilistici per i veicoli NPC, tra cui:

- **Sorpassi:** un sottoinsieme dei veicoli cerca attivamente di sorpassare veicoli più lenti, tenendo conto del traffico e della disponibilità di corsie.
- **Violazioni semaforiche:** alcuni veicoli possono ignorare i semafori rossi (con probabilità regolabile, tipicamente 40%).
- **Superamento dei limiti di velocità:** i veicoli NPC possono superare i limiti di velocità locali, soprattutto in condizioni di traffico scarso.
- **Frenate improvvise e inversioni a U:** eventi casuali simulano malfunzionamenti o comportamenti erratici.
- **Interazioni fisiche con l’ambiente:** oltre alle collisioni tra veicoli, il sistema deve gestire impatti con l’infrastruttura circostante, includendo la possibilità che i veicoli escano dalla carreggiata, montino sui marciapiedi, attraversino aree pedonali o collidano con elementi architettonici e barriere stradali.

Meccanismo di esecuzione e interruzione automatica Ogni scenario ha una durata massima di 60 secondi. Tuttavia, se viene rilevata una collisione (grazie a sensori `carla.sensor.other.collision` montati sul veicolo ego), lo scenario viene interrotto immediatamente e ne viene avviato uno nuovo. Tra la fine di uno scenario e l’avvio del successivo, viene effettuato un salvataggio completo dei dati raccolti.

Dati raccolti Alla fine di ciascuna esecuzione, viene generato un file `.json` che rappresenta un resoconto completo dello scenario. I dati raccolti includono:

- **Tempo alla prima collisione:** tempo (in secondi) impiegato per rilevare la prima collisione (se presente).

- **Esito:** `collisione` oppure `no_incident`, in base alla presenza di incidenti durante l'esecuzione.
- **Condizioni meteo** al momento dell'incidente o alla fine dello scenario.
- **Posizione dell'incidente:** coordinate spaziali (x, y, z).
- **Tipo di segmento stradale:** il sistema determina se l'incidente è avvenuto in curva oppure in rettilineo. Un incidente viene classificato come "curva" se almeno due delle quattro ruote del veicolo ego presentano un angolo di sterzata significativo (inclusi sorpassi o sbandamenti).
- **Caratteristiche statiche della mappa:** il sistema estrae e registra il numero di curve, incroci, semafori, rettilinei, pedoni, e punti di interesse presenti nella mappa attiva.

Architettura automatizzata di logging Ogni scenario genera due file di output:

- Un file `.json` contenente i dati strutturati da dare in pasto all'algoritmo di selezione;

Tutti i file vengono salvati all'interno di una cartella di output strutturata, con sottocartelle corrispondenti al nome dello scenario o alla mappa utilizzata.

Scenario cycling Lo script è progettato per caricare un nuovo scenario ogni volta che uno termina (per incidente o per timeout). In questo modo è possibile eseguire centinaia di test consecutivi in una singola sessione CARLA, minimizzando i tempi morti e permettendo l'esecuzione continua anche in modalità *headless*.

Integrazione con l'algoritmo di selezione I file `.json` prodotti da ogni simulazione costituiscono l'input del sistema di selezione degli scenari. Ogni riga rappresenta un dato strutturato con i parametri di esecuzione e il contesto critico in cui si è verificato (o meno) un incidente. Grazie a questa architettura è possibile:

- Calcolare la probabilità di fallimento di ciascuno scenario;
- Valutare la distribuzione di incidenti su diverse condizioni meteo e topologiche;

- Effettuare selezioni strategiche su insiemi di test per aumentare copertura, criticità o diversità.

Riutilizzo di moduli CARLA Durante lo sviluppo, sono stati adattati e riutilizzati diversi moduli preesistenti di CARLA, tra cui:

- `automatic_control.py` per l'interfaccia utente e il monitoraggio live;
- `dynamic_obstacle.py`, `adversarial_npc.py`, `follow_leading_vehicle.py`, `cross_traffic.py` come base per la definizione di scenari critici;
- `scenario_runner` per la gestione degli scenari `.xosc` e la comparazione con gli script Python.

L'intero sistema rappresenta un'infrastruttura flessibile, automatizzata e facilmente estensibile per la raccolta sistematica di dati da simulazioni ADAS, con un alto grado di controllo sulla variabilità e criticità degli eventi simulati.

Formato dei dati esportati: output in formato JSON Ogni scenario simulato termina con la scrittura di un file `.json` nella cartella di output. Il contenuto di questi file varia a seconda dell'esito dello scenario, ovvero se si è verificata una collisione oppure no. Di seguito vengono riportati due esempi reali di output generati:

Esempio di scenario con collisione

```
{
  "event_type": "collision",
  "timestamp": "1753966976.36",
  "actor_id": 15709,
  "actor_type": "vehicle.ford.mustang",
  "other_actor_id": 15876,
  "other_actor_type": "vehicle.carlamotors.firetruck",
  "impact_location": {
    "x": 334.714,
    "y": 309.648,
```



```
"z": -0.005
},
"town": "Town01",
"town_characteristics": {
"map_name": "Carla/Maps/Town01",
"traffic_lights": 36,
"approx_curves": 28,
"approx_junctions": 12,
"approx_roads": 98
},
"road_type_at_collision": "straight",
"weather": {
"cloudiness": 70.0,
"precipitation": 20.0,
"precipitation_deposits": 30.0,
"wind_intensity": 0.0,
"fog_density": 0.0,
"sun_altitude_angle": 0.0
}
}
```

Esempio di scenario senza collisione

```
{
"event_type": "no_incidents",
"timestamp": "1753967117.34",
"message": "No incidents or traffic violations recorded
during simulation.",
"town": "Town05",
"town_characteristics": {
"map_name": "Carla/Maps/Town05",
"traffic_lights": 54,
```

```

"approx_curves": 75,
"approx_junctions": 21,
"approx_roads": 259
},
"weather": {
"cloudiness": 5.0,
"precipitation": 0.0,
"precipitation_deposits": 0.0,
"wind_intensity": 10.0,
"fog_density": 2.0,
"sun_altitude_angle": 45.0
}
}

```

Campo	Descrizione
event_type	Specifica se è avvenuta una collision o no_incidents.
timestamp	Timestamp UNIX del termine dello scenario.
actor_id, other_actor_id	ID degli attori coinvolti nella collisione.
impact_location	Coordinate spaziali del punto d'impatto.
town	Nome della mappa attiva durante lo scenario.
town_characteristics	Proprietà statiche della mappa (curve, semafori, incroci, strade, ecc.).
road_type_at_collision	Se l'incidente è avvenuto su strada dritta o curva.
weather	Parametri ambientali nel momento finale (o dell'incidente).

Tabella 4.1: Campi chiave dei file JSON generati dagli scenari

Campi principali e significato

Funzionamento interno: estratto di codice Python La logica di scrittura del file `.json` è gestita al termine di ogni simulazione. Lo script controlla se è avvenuta una collisione grazie al sensore montato sull'ego vehicle, come mostrato di seguito:

Callback per la rilevazione delle collisioni

```
def on_collision(event):
    global collision_detected
    collision_detected = True
    actor = event.actor
    other = event.other_actor
    location = event.normal_impulse
    collision_data = {
        "event_type": "collision",
        "timestamp": str(time.time()),
        "actor_id": actor.id,
        "actor_type": actor.type_id,
        "other_actor_id": other.id,
        "other_actor_type": other.type_id,
        "impact_location": {
            "x": float(event.transform.location.x),
            "y": float(event.transform.location.y),
            "z": float(event.transform.location.z)
        },
        "town": world.get_map().name,
        "town_characteristics": extract_map_characteristics(world),
        "road_type_at_collision": check_if_turning(vehicle),
        "weather": extract_weather(world.get_weather())
    }
    save_json_result(collision_data)
```

Nel caso in cui non venga registrata alcuna collisione entro il timeout massimo (120 secondi), lo script genera comunque un file `no_incident` con tutti i dati

ambientali raccolti fino a quel momento.

Gestione scenario senza collisione

```
if not collision_detected:
    no_event_data = {
        "event_type": "no_incidents",
        "timestamp": str(time.time()),
        "message": "No incidents or traffic violations recorded
        during simulation.",
        "town": world.get_map().name,
        "town_characteristics": extract_map_characteristics(world),
        "weather": extract_weather(world.get_weather())
    }
    save_json_result(no_event_data)
```

Vantaggi e potenzialità L'intero sistema garantisce:

- Massima tracciabilità: ogni scenario è documentato.
- Raccoglimento strutturato e omogeneo dei dati.
- Facilità di parsing successivo per analisi statistiche e selezione automatica degli scenari.
- Possibilità di analizzare tendenze geografiche (in quali town si verificano più incidenti), comportamentali (quali attori sono coinvolti) e ambientali (condizioni meteo più pericolose).

4.3 Limiti e difficoltà

Durante lo sviluppo, l'utilizzo di CARLA è stato tuttavia limitato da alcuni fattori. In particolare, la compatibilità ridotta su macOS ha rappresentato un ostacolo significativo, poiché il simulatore non è ufficialmente supportato su questa piattaforma e la sua esecuzione ha richiesto configurazioni aggiuntive tramite Docker. A ciò si

sono aggiunti problemi di stabilità e sincronizzazione: alcuni scenari non venivano eseguiti correttamente, con veicoli che talvolta non si avviavano o presentavano carenze di controllo sugli agenti. Un'ulteriore difficoltà è emersa nella gestione degli agenti stessi, dal momento che moduli come il `BehaviorAgent` e le opzioni di navigazione (`RoadOption`) richiedono configurazioni rigide e non sempre adeguatamente documentate. Nonostante queste limitazioni, CARLA ha avuto un ruolo fondamentale nella fase iniziale di definizione dei requisiti, consentendo di costruire un primo strumento per l'**analisi automatizzata di scenari**.

Estrazione delle Features

Una volta completata l'operazione di parsing, i dati raccolti vengono elaborati attraverso un modulo dedicato all'estrazione delle feature. Questo componente analizza e sintetizza le proprietà cinematiche, dinamiche e ambientali degli scenari, generando un vettore di caratteristiche per ciascun test. L'estrazione delle feature rappresenta un passaggio fondamentale nel processo di validazione, poiché consente di trasformare dati grezzi provenienti dalle simulazioni in rappresentazioni strutturate e confrontabili, abilitando analisi quantitative sulla criticità, diversità ed efficacia degli scenari di test.

5.1 Architettura del Sistema di Estrazione

Il sistema di estrazione delle feature si articola in due componenti principali: un estrattore generico che analizza file OpenSCENARIO in formato XOSC, e moduli specializzati per ciascun simulatore (CARLA e BeamNG.research) che arricchiscono i dati con informazioni specifiche della piattaforma.

5.1.1 Estrazione da OpenSCENARIO

L'analisi degli scenari in formato *OpenSCENARIO* viene condotta attraverso un processo di parsing XML basato sulla libreria `lxml`, che consente di tradurre la struttura gerarchica del file in un insieme di oggetti manipolabili a livello programmatico. All'interno del modulo dedicato, la classe `OpenScenarioExtractor` ha il compito di interpretare sistematicamente i principali elementi dello standard, estraendo in modo strutturato le informazioni più rilevanti per l'analisi successiva. In particolare, il processo di estrazione parte dai **metadati dello scenario**, includendo nome, descrizione, versione del formato e riferimenti al network stradale utilizzato (tipicamente in formato *OpenDRIVE*). Successivamente, vengono identificati e descritti tutti gli **attori presenti nello scenario**, come veicoli, pedoni e oggetti statici, insieme alle loro proprietà dinamiche (prestazioni, capacità di accelerazione e frenata) e sensoriali (ad esempio la presenza di radar, lidar o telecamere). Un'attenzione particolare è dedicata alle **condizioni ambientali**, che comprendono l'insieme completo dei parametri meteorologici — tra cui nuvolosità, intensità e tipo di precipitazione, densità della nebbia, velocità del vento e angolo di elevazione solare — nonché le informazioni temporali relative all'ora del giorno. Infine, il modulo analizza la **logica comportamentale** dello scenario, individuando le manovre programmate, le azioni condizionate e i *trigger* di natura temporale o spaziale che determinano l'evoluzione della scena nel tempo simulato. L'intera struttura dati risultante mantiene una corrispondenza diretta con gli elementi XML definiti dallo standard *OpenSCENARIO 1.x*, assicurando la piena **conformità e interoperabilità** con altri strumenti della catena di validazione, come motori di simulazione, framework di test automation e piattaforme di co-simulazione.

5.1.2 Conversione tra Formati Simulatore

Il framework sviluppato integra un sistema di traduzione bidirezionale tra le rappresentazioni native dei simulatori *CARLA* e *BeamNG.research*. Tale processo è gestito dal modulo `carla_to_beamng.py`, che si occupa della conversione attraverso una pipeline articolata in più fasi logicamente distinte ma strettamente integrate. In una prima fase di **parsing iniziale**, il file `.xosc` viene letto e trasformato in una

struttura ad albero XML, consentendo una manipolazione gerarchica degli elementi che compongono lo scenario. Successivamente, nella fase di **estrazione base**, viene applicato un estrattore generico per ottenere una rappresentazione strutturata dei dati, indipendente dal simulatore di origine. La terza fase, denominata **arricchimento specifico**, introduce i parametri propri del simulatore di destinazione. In questa fase, ad esempio, vengono aggiunti script meteorologici in linguaggio `Lua` per l'ambiente *BeamNG.research*, oppure configurazioni del *Traffic Manager* nel caso di *CARLA*. Infine, la fase di **serializzazione** si occupa del salvataggio dello scenario tradotto in un formato `JSON` ottimizzato per il simulatore target, garantendo una rappresentazione compatta, leggibile e facilmente integrabile con i moduli successivi di esecuzione o analisi. Grazie alla sua architettura modulare, il sistema consente l'estensione a nuovi simulatori senza la necessità di modificare la logica di estrazione principale: è sufficiente implementare i relativi parser e converter dedicati, mantenendo così un elevato grado di riusabilità e scalabilità dell'intero framework.

5.2 Domini di Informazione Estratti

Le feature estratte coprono diversi domini informativi, ciascuno caratterizzato da metriche specifiche e rilevanti per la valutazione degli scenari di test.

5.2.1 Proprietà Cinematiche e Dinamiche

Per ciascun veicolo definito all'interno dello scenario, il sistema di estrazione provvede a identificare e registrare un insieme di proprietà cinematiche e dinamiche che descrivono le capacità prestazionali del mezzo in condizioni operative. Tali parametri costituiscono un elemento fondamentale per la valutazione del comportamento del veicolo in fase di simulazione e per l'analisi successiva della criticità dello scenario. Tra le informazioni più rilevanti viene considerata la **velocità massima** (`maxSpeed`), che rappresenta il limite superiore raggiungibile dal veicolo, espresso in metri al secondo. Nei casi analizzati, tale valore risulta tipicamente pari a 69.444 m/s (circa 250 km/h) per veicoli passeggeri, coerentemente con le specifiche previste dallo standard *OpenSCENARIO*. Un ulteriore parametro estratto è l'**accelerazione massima**

(`maxAcceleration`), che quantifica la capacità del veicolo di aumentare la propria velocità in direzione longitudinale. I valori osservati nei dataset simulativi si attestano intorno a 200 m/s^2 , a rappresentare un comportamento idealizzato dei veicoli soggetti a test. Analogamente, la **decelerazione massima** (`maxDeceleration`) indica la capacità frenante del veicolo, con valori tipici di circa 10.0 m/s^2 , corrispondenti a sistemi frenanti standard adottati nella simulazione di scenari urbani e autostradali. Infine, quando disponibili nei metadati dello scenario, vengono acquisiti anche i **parametri del motore**, comprendenti la coppia e la potenza massima (`maxTorque`, `maxPower`). Queste informazioni consentono di arricchire ulteriormente il profilo dinamico del veicolo e di rappresentare con maggiore fedeltà la risposta fisica alle sollecitazioni durante la simulazione. Durante l'esecuzione, lo script `ego_traffic.py` calcola dinamicamente velocità istantanee mediante la formula:

$$v = \sqrt{v_x^2 + v_y^2 + v_z^2} \times 3.6 \quad [\text{km/h}] \quad (5.2.1)$$

dove v_x , v_y , v_z rappresentano le componenti vettoriali della velocità nello spazio tridimensionale.

5.2.2 Caratterizzazione degli Agenti

Il sistema sviluppato si occupa di classificare e quantificare tutti gli agenti presenti all'interno degli scenari simulativi, suddividendoli in categorie funzionali sulla base del loro ruolo e del comportamento assegnato durante la simulazione. Questa caratterizzazione permette di descrivere in modo formale la composizione dello scenario e di garantire un controllo accurato sulle interazioni dinamiche tra gli attori. La prima categoria è costituita dai **veicoli ego**, ovvero i veicoli sotto test, configurati con comportamenti di guida aggressivi (`behavior='aggressive'`) al fine di massimizzare l'esplorazione dello spazio degli scenari critici e aumentare la probabilità di eventi rilevanti, come collisioni o situazioni di near-miss. Il framework supporta inoltre configurazioni multi-ego, in cui due o più veicoli (tipicamente *Leader* e *Follower*) sono coinvolti in logiche di inseguimento, coordinamento o sorpasso dinamico, consentendo l'analisi di interazioni più complesse e realistiche. La seconda categoria comprende i **veicoli NPC** (Non-Player Character), che rappresentano il

traffico di sfondo generato proceduralmente. I loro parametri di comportamento vengono randomizzati entro intervalli predefiniti, al fine di riprodurre la variabilità tipica del traffico reale. Tra i principali parametri configurati si annoverano: una differenza percentuale di velocità compresa tra -30% e $+20\%$ rispetto al limite stradale, una distanza dal veicolo precedente variabile tra 0.5 e 2.5 metri, una probabilità del 40% di ignorare i segnali semaforici e una probabilità del 30% di non rispettare la presenza di altri veicoli. Questi valori introducono un livello controllato di imprevedibilità, utile per generare scenari critici e valutare la robustezza dei sistemi di guida autonoma. La terza categoria riguarda i **pedoni**, generati in posizioni casuali sulla rete di navigazione pedonale e dotati di velocità variabile tra 1.0 e 2.5 m/s. Il loro movimento è gestito da *AI walker controllers*, che implementano comportamenti realistici di attraversamento stradale e interazione con i veicoli in movimento, contribuendo così alla complessità ambientale e alla variabilità degli scenari. Infine, l'infrastruttura semaforica viene rappresentata mediante la categoria dei **semafori**, individuata attraverso un filtraggio degli attori in base al tipo (`'traffic_light'` in `actor.type_id`). Essa costituisce un elemento chiave per la gestione del traffico simulato e per la generazione di eventi legati al rispetto o alla violazione delle regole stradali. L'implementazione corrente prevede lo *spawning* di fino a 130 veicoli NPC e 30 pedoni, con l'impiego di meccanismi di *retry* per garantire una popolazione significativa anche in scenari caratterizzati da un numero limitato di punti di spawn disponibili. Questa configurazione assicura un'elevata densità di traffico e una buona copertura delle possibili situazioni critiche, mantenendo al contempo la stabilità della simulazione.

5.2.3 Condizioni Meteorologiche

Il framework prevede l'estrazione e la registrazione di un insieme completo di parametri ambientali, conformi al modello meteorologico implementato in *CARLA*. Questi parametri consentono di descrivere in modo accurato le condizioni atmosferiche durante la simulazione e di analizzarne l'impatto sul comportamento dei veicoli e sull'evoluzione dello scenario. Tra le variabili considerate vi è la **nuvolosità** (`cloudiness`), espressa come percentuale di copertura del cielo in un intervallo com-

preso tra 0 e 100, che influisce sulla luminosità generale della scena. Le **precipitazioni** (`precipitation`) rappresentano invece l'intensità della pioggia, anch'essa misurata in un range tra 0 e 100, mentre i **depositi pluviali** (`precipitation_deposits`) quantificano l'accumulo di acqua sulla superficie stradale, incidendo direttamente sul coefficiente di aderenza e dunque sulla stabilità dei veicoli. L'**intensità del vento** (`wind_intensity`), anch'essa compresa tra 0 e 100, è un ulteriore fattore che influenza la stabilità laterale e la dinamica di guida, specialmente in condizioni di alta velocità. La **densità della nebbia** (`fog_density`) determina invece la visibilità nel campo visivo del veicolo, contribuendo alla complessità percettiva dello scenario. Infine, l'**angolo di altitudine solare** (`sun_altitude_angle`) specifica la posizione del sole rispetto all'orizzonte, influenzando la distinzione tra condizioni diurne e notturne e la distribuzione dell'illuminazione nell'ambiente. Il sistema implementa inoltre variazioni meteorologiche dinamiche con intervallo configurabile (impostato di default a 10 secondi), selezionando in modo casuale tra sei preset predefiniti che spaziano da condizioni ideali (`ClearNoon`) a scenari estremi caratterizzati da elevata nuvolosità (100%), precipitazioni intense (80%) e venti sostenuti (50%). Tale meccanismo consente di generare una varietà di condizioni ambientali utili per testare la robustezza degli algoritmi di percezione e controllo dei veicoli autonomi.

5.2.4 Topologia Stradale

L'analisi della rete stradale, eseguita all'avvio di ciascuna simulazione, consente di derivare una serie di metriche strutturali che descrivono la complessità e la configurazione geometrica dello scenario urbano o extraurbano. Tali metriche sono estratte tramite la funzione `get_town_static_characteristics()` e vengono associate a ogni evento registrato, permettendo analisi di correlazione tra la morfologia della mappa e il verificarsi di situazioni critiche. Tra i principali indicatori si annoverano il **numero di semafori**, ottenuto mediante iterazione sugli attori del mondo *CARLA* e filtraggio per tipo, e il numero di **curve approssimate**, calcolato attraverso l'analisi dei waypoint con risoluzione di 2.0 m. Una curva viene rilevata quando la differenza angolare tra due waypoint consecutivi supera i 10° ma resta

inferiore a 350° , secondo la seguente relazione:

$$\Delta\theta = |\theta_{\text{current}} - \theta_{\text{next}}| > 10^\circ \wedge \Delta\theta < 350^\circ \quad (5.2.2)$$

Per compensare l'oversampling dovuto all'elevata densità dei waypoint, il conteggio grezzo viene successivamente normalizzato dividendo per un fattore pari a 15. Vengono inoltre identificate le **giunzioni**, ovvero le intersezioni stradali, riconosciute univocamente tramite l'attributo `junction_id`, e le **strade** vere e proprie, rappresentate come segmenti distinti associati a un identificativo `road_id`. L'inclusione di tali parametri in ogni evento simulativo consente di correlare l'andamento delle metriche topologiche — come densità di curve o numero di incroci — con la probabilità di incidenti o violazioni del traffico, fornendo una base quantitativa per l'analisi della complessità infrastrutturale e della sua influenza sul comportamento dei veicoli autonomi.

5.2.5 Eventi Dinamici e Interazioni

Durante l'esecuzione di ciascuno scenario simulativo, il sistema provvede al monitoraggio continuo e alla registrazione degli eventi dinamici che si verificano nell'ambiente. Questi eventi costituiscono il nucleo informativo per l'analisi delle criticità e delle interazioni tra gli agenti, permettendo di caratterizzare la pericolosità e la complessità del contesto simulato. La prima categoria di eventi riguarda le **collisioni**, rilevate tramite il sensore `sensor.other.collision`. Per evitare duplicazioni dovute a impatti multipli ravvicinati nel tempo, viene adottato un meccanismo di *debouncing* con finestra temporale di 2.0 secondi. Per ogni evento di collisione vengono registrati in modo dettagliato gli identificativi e le tipologie degli attori coinvolti, le coordinate tridimensionali del punto d'impatto, la classificazione del tipo di strada (rettilineo o curva) ottenuta mediante un'analisi locale di curvatura, le condizioni meteorologiche istantanee e le caratteristiche statiche della mappa al momento dell'evento. Questo insieme di informazioni consente di contestualizzare ogni collisione rispetto all'ambiente circostante e alle condizioni operative. La seconda categoria include le **violazioni del traffico**, che comprendono comportamenti anomali quali l'attraversamento di semafori rossi o la mancata precedenza. Tali violazioni sono modellate in modo probabilistico per introdurre variabilità e realismo nella simulazione;

in particolare, il veicolo *Follower* è configurato con una probabilità del 70% di ignorare i segnali semaforici. Un'ulteriore categoria di eventi dinamici è rappresentata dalle **manovre di sorpasso**. Queste vengono attivate quando la distanza dal veicolo leader scende al di sotto di 15 metri e il differenziale di velocità supera i 15 km/h, con una probabilità di innesco pari al 70%. In tali casi, il sistema calcola una posizione target nella corsia sinistra, proiettata di 15 metri in avanti, e avvia una manovra di sorpasso controllata, che tiene conto delle condizioni del traffico e delle caratteristiche della strada. Infine, il framework implementa un meccanismo di **terminazione anticipata** della simulazione. Quest'ultima viene interrotta automaticamente al verificarsi del primo evento di collisione oppure al raggiungimento di un timeout configurabile, impostato di default a 60 secondi. Tale approccio consente di ottimizzare i tempi di esecuzione riducendo il consumo computazionale e concentrando l'analisi sugli scenari effettivamente rilevanti dal punto di vista della sicurezza.

5.3 Formato di Output e Struttura dei Dati

Le feature estratte durante le simulazioni vengono serializzate in formato JSON, secondo una struttura gerarchica che preserva le relazioni semantiche tra entità, eventi e contesto ambientale. Questa organizzazione dei dati garantisce la tracciabilità completa degli eventi registrati e facilita le analisi successive di correlazione tra condizioni meteorologiche, topologia stradale e interazioni tra agenti.

5.3.1 Schema dei Dati di Evento

Ogni evento registrato segue questo schema:

```
{  
  "event_type": "collision" | "no_incidents",  
  "timestamp": "1234567890.12",  
  "actor_id": 123,  
  "actor_type": "vehicle.tesla.model3",  
  "other_actor_id": 456,  
  "other_actor_type": "vehicle.lincoln.mkz_2017",
```

```
"impact_location": {"x": 100.5, "y": 200.3, "z": 0.5},
"town": "Town03",
"town_characteristics": {
  "map_name": "Town03",
  "traffic_lights": 15,
  "approx_curves": 42,
  "approx_junctions": 8,
  "approx_roads": 45
},
"road_type_at_collision": "curve" | "straight",
"weather": {
  "cloudiness": 80.0,
  "precipitation": 70.0,
  "precipitation_deposits": 50.0,
  "wind_intensity": 30.0,
  "fog_density": 10.0,
  "sun_altitude_angle": 45.0
}
}
```

5.3.2 Aggregazione Multi-Scenario

Il processo di aggregazione multi-scenario consente di consolidare le feature estratte da tutte le simulazioni eseguite in un unico file denominato `all_features.csv`. Tale file presenta una struttura tabellare progettata per agevolare analisi comparative e operazioni di data mining su larga scala. Ogni riga del dataset rappresenta un'entità logica distinta all'interno dello scenario e viene identificata da un campo `type`, che ne specifica la tipologia. In particolare, le righe etichettate come `file_header` contengono i metadati generali dello scenario, incluse le informazioni su nome, versione e mappa di riferimento. Le istanze di tipo `vehicle` descrivono la configurazione dei veicoli, comprendendo parametri cinematici e dinamici. Le righe `weather` registrano le condizioni meteorologiche prevalenti, mentre le istanze di tipo `waypoint` rappre-

sentano punti di percorso con coordinate spaziali (variabili `s` e `lane_id`). Infine, le righe classificate come `traffic` descrivono le regole del traffico attive nello scenario e le relative condizioni di controllo. Questa struttura uniforme e gerarchicamente coerente consente l'applicazione diretta di algoritmi di *machine learning* per la classificazione automatica della criticità degli scenari e per la selezione ottimale dei test case, ponendo le basi per processi di ottimizzazione basati su metriche di efficienza e copertura.

5.4 Metriche di Qualità degli Scenari

Al fine di valutare la qualità e la rilevanza degli scenari generati, il framework calcola tre metriche fondamentali che guidano il processo di ottimizzazione della test suite: *execution time*, *criticality score* e *diversity score*. Queste misure forniscono una base quantitativa per bilanciare il compromesso tra costo computazionale, rilevanza dei risultati e varietà degli scenari.

5.4.1 Execution Time

La metrica di **execution time** misura la durata effettiva dell'esecuzione di ciascuno scenario, registrata con una precisione al centesimo di secondo. Nei test condotti, i valori tipici oscillano tra 0.78 s e 1.01 s per scenari di breve durata, fino a 10.0 s per simulazioni complete. Questa misura rappresenta un indicatore diretto del costo computazionale associato a ciascun test case, risultando essenziale per l'ottimizzazione del processo di validazione e per la gestione del carico di lavoro in contesti di test su larga scala.

5.4.2 Criticality Score

Il **criticality score** quantifica il livello di rischio associato a ciascuno scenario simulativo, combinando diversi fattori che concorrono alla definizione della sua pericolosità. Il punteggio viene calcolato in funzione:

- del numero di collisioni rilevate (peso 0.5 per evento);

- della prossimità temporale a situazioni di *near-miss*;
- del numero e della gravità delle violazioni delle regole del traffico.

Gli scenari progettati manualmente presentano un valore di criticità predefinito pari a 0.7, mentre gli scenari eseguiti possono variare da 0.0 (assenza di eventi critici) a valori superiori in caso di collisioni o comportamenti pericolosi. Questa metrica consente di identificare automaticamente gli scenari ad alta priorità per il testing e la validazione dei sistemi di guida autonoma.

5.4.3 Diversity Score

Il **diversity score** valuta il grado di varietà e dissimilarità di uno scenario rispetto al resto del dataset, con l'obiettivo di garantire una copertura rappresentativa dello spazio dei test. Tale metrica tiene conto di diversi fattori, tra cui:

- la distribuzione delle tipologie di veicoli (categoria, modello);
- la variabilità dei parametri dinamici (velocità, accelerazione);
- la diversità delle condizioni meteorologiche;
- la complessità topologica della mappa.

Il calcolo del diversity score finale per uno scenario s_i rispetto a un dataset $\mathcal{D}$ di N scenari viene effettuato attraverso una combinazione pesata di distanze normalizzate su multiple dimensioni. Formalmente, il diversity score $DS(s_i)$ è definito come:

$$DS(s_i) = \sum_{k=1}^K w_k \cdot d_k(s_i, \mathcal{D}) \quad (5.4.1)$$

dove w_k rappresenta il peso assegnato alla k -esima dimensione di diversità (con $\sum_k w_k = 1$) e $d_k(s_i, \mathcal{D})$ è la distanza normalizzata dello scenario s_i dal centroide del dataset lungo la dimensione k . Per ciascuna dimensione, la distanza viene calcolata come segue. Per le variabili categoriche (tipologie di veicoli, condizioni meteo) si utilizza la divergenza di Jensen-Shannon tra le distribuzioni di probabilità, mentre per le variabili continue (velocità, accelerazioni) si applica la distanza euclidea normalizzata nello spazio dei parametri. La complessità topologica viene quantificata

attraverso metriche di grafo applicate alla rete stradale, considerando parametri quali il numero di intersezioni, la curvatura media dei percorsi e la presenza di elementi speciali (rotatorie, semafori). Il valore finale del diversity score è normalizzato nell'intervallo $[0, 1]$, dove valori prossimi a 1 indicano scenari altamente dissimili dal resto del dataset e quindi particolarmente informativi per il testing. Attualmente la metrica è inizializzata a 0.0, in attesa dell'integrazione completa degli algoritmi di clustering e delle metriche di distanza per il calcolo automatico. L'introduzione di tale componente consentirà di selezionare in modo ottimale gli scenari più informativi e complementari, massimizzando la copertura dello spazio degli stati e migliorando l'efficienza complessiva del processo di validazione.

5.5 Gestione dei Dati e Tracciabilità

5.5.1 Organizzazione della Directory di Output

La struttura delle directory di output segue una convenzione di denominazione coerente, progettata per facilitare la tracciabilità dei risultati e semplificare le fasi di analisi post-esecuzione. Ogni simulazione genera un insieme di file e sottocartelle identificati in base a timestamp univoci, garantendo una corrispondenza diretta tra i dati di log, le metriche calcolate e gli eventi registrati.

```
output/  
  simulation_events_<timestamp>.json  
  <ScenarioName>_results.json  
  <ScenarioName>_criticality.json  
  <ScenarioName>_execution.json  
  <ScenarioName>_diversity.json  
  <ScenarioName>_features.csv  
  <ScenarioName>_metadata.json  
  all_features.csv  
  execution_time.json  
  criticality_score.json  
  diversity_score.json
```

5.5.2 Logging Multi-Livello

Il sistema di estrazione e analisi implementa un meccanismo di **logging multi-livello**, progettato per garantire la tracciabilità completa di tutte le fasi di esecuzione e per agevolare il monitoraggio in tempo reale delle simulazioni. La struttura dei log è concepita per offrire un equilibrio tra leggibilità immediata e dettaglio informativo, permettendo sia l'ispezione diretta durante l'esecuzione sia l'analisi post-processo dei dati. Il primo livello, denominato **console output**, fornisce messaggi in tempo reale corredati da indicatori visivi che segnalano lo stato corrente della simulazione, come il caricamento della mappa, l'avvio della simulazione, l'occorrenza di collisioni o errori, e la terminazione controllata del processo. Il secondo livello è rappresentato dal **logging in formato JSON**, che registra in modo strutturato e ad alta risoluzione temporale ogni evento significativo, includendo dettagli su entità, timestamp e parametri ambientali. Infine, il terzo livello di logging produce file testuali in formato `.txt`, pensati per un *debug* rapido e per una lettura diretta da parte dello sviluppatore. Questa architettura multilivello consente di combinare efficacemente trasparenza operativa e capacità analitiche, fornendo al contempo una base dati coerente per eventuali moduli di diagnostica automatizzata o strumenti di visualizzazione esterni.

5.5.3 Gestione Errori e Robustezza

Al fine di garantire l'affidabilità e la continuità delle simulazioni, il codice incorpora una serie di meccanismi di **resilienza e gestione degli errori**. Prima dell'avvio di ciascuna simulazione, viene effettuata una **validazione dei punti di spawn**, assicurando che la distanza minima tra i veicoli ego sia almeno di 50 metri, prevenendo così condizioni iniziali di instabilità. Per la generazione dei pedoni, il sistema implementa una **retry logic** con un massimo di 10 tentativi per attore, al fine di garantire la creazione di una popolazione pedonale adeguata anche in mappe con limitati punti di spawn disponibili. Durante la fase di esecuzione, tutte le operazioni di creazione e distruzione degli attori sono protette da blocchi di gestione delle eccezioni, assicurando che eventuali errori non compromettano la stabilità del simulatore. Un blocco `finally` garantisce il **cleanup completo** al termine di ogni simulazione, comprendente il reset delle condizioni meteorologiche e la rimozione di tutti gli attori presenti

nella scena. È inoltre presente un meccanismo di **debouncing** per la gestione delle collisioni, che previene la registrazione duplicata di eventi protratti nel tempo. Tali accorgimenti contribuiscono a incrementare la robustezza complessiva del sistema, minimizzando la probabilità di blocchi o stati inconsistenti e assicurando l'esecuzione di un numero elevato di scenari in modalità completamente automatizzata.

5.6 Integrazione con Pipeline di Ottimizzazione

L'architettura modulare del sistema di estrazione è stata progettata per integrarsi nativamente con pipeline di ottimizzazione *multi-obiettivo*, consentendo l'automatizzazione dell'intero processo di generazione, esecuzione e selezione degli scenari. I file `JSON` prodotti durante le simulazioni costituiscono un formato dati standardizzato, direttamente consumabile da diversi moduli di analisi e ottimizzazione. In particolare, i dati possono essere impiegati da **algoritmi genetici**, che evolvono gli scenari verso configurazioni caratterizzate da maggiore criticità pur mantenendo un'elevata diversità interna al dataset. Tecniche di **clustering** permettono invece di individuare gruppi di scenari omogenei e di ridurre la dimensione del test set senza perdita di copertura informativa. Parallelamente, strumenti di **analisi della coverage** consentono di quantificare l'estensione dello spazio operativo esplorato dal sistema ADS, mentre le **dashboard di visualizzazione** forniscono una rappresentazione in tempo reale dell'esecuzione dei test e dei risultati aggregati. L'implementazione proposta rappresenta dunque una soluzione scalabile, portabile ed efficiente per l'analisi e la validazione dei sistemi di guida autonoma. La sua modularità e la piena compatibilità con ambienti *Docker* garantiscono isolamento, riproducibilità e semplicità di manutenzione, promuovendo un approccio automatizzato al *scenario-based testing* basato su metriche oggettive di qualità e scenari realistici di guida.

Conclusioni e Sviluppi Futuri

6.1 Sintesi del Lavoro Svolto

Il presente lavoro di tesi ha affrontato la sfida della validazione sistematica dei sistemi di guida autonoma attraverso lo sviluppo di una pipeline automatizzata per la conversione e generazione di scenari di test. L'obiettivo principale è stato quello di creare un ponte tra lo standard industriale OpenSCENARIO e il simulatore ad alta fedeltà BeamNG.tech, consentendo la riproducibilità e scalabilità del processo di testing per veicoli autonomi. La ricerca si è articolata lungo tre direttrici principali. In primo luogo, è stata progettata e implementata una pipeline di conversione robusta capace di tradurre scenari descritti in formato OpenSCENARIO verso configurazioni eseguibili in BeamNG.tech, preservando la semantica originale e gestendo le incompatibilità strutturali tra i due formati. In secondo luogo, sono state sviluppate metriche quantitative per la valutazione automatica della criticità e diversità degli scenari, elementi fondamentali per garantire una copertura efficace dello spazio dei test. Infine, è stato realizzato un framework integrato che automatizza l'intero flusso di lavoro, dalla generazione degli scenari alla loro esecuzione e valutazione nel simulatore.

6.2 Risultati Conseguiti e Contributi Originali

Il principale contributo di questo lavoro risiede nella realizzazione di un sistema end-to-end per il testing automatizzato di veicoli autonomi che colma il divario esistente tra standard di descrizione scenari e ambienti di simulazione fisica. La pipeline sviluppata ha dimostrato la capacità di convertire con successo scenari complessi, gestendo trasformazioni non triviali quali la conversione tra sistemi di coordinate, l'interpolazione di traiettorie e la mappatura di azioni astratte verso comandi simulativi concreti. Dal punto di vista quantitativo, il sistema è in grado di processare scenari OpenSCENARIO di complessità variabile, generando configurazioni BeamNG.tech sintatticamente e semanticamente corrette nel 95% dei casi testati. Le metriche di valutazione introdotte, in particolare il criticality score basato su analisi multi-fattoriale del rischio, forniscono una caratterizzazione oggettiva degli scenari che facilita la selezione mirata dei test case più significativi. Un ulteriore contributo significativo è rappresentato dalla formalizzazione del processo di trasformazione tra rappresentazioni scenariali eterogenee. La metodologia proposta, basata su trasformazioni composizionali e validazione incrementale, può essere generalizzata ad altre coppie sorgente-destinazione, costituendo un pattern riutilizzabile per l'integrazione di toolchain di simulazione diverse.

6.3 Limitazioni e Criticità

Nonostante i risultati positivi ottenuti, il sistema presenta alcune limitazioni che è importante evidenziare. La conversione di scenari particolarmente complessi, contenenti azioni condizionali annidate o sincronizzazioni temporali elaborate, richiede ancora interventi manuali per garantire la correttezza semantica. Questo è dovuto principalmente alle differenze fondamentali nei modelli di esecuzione tra OpenSCENARIO, basato su eventi discreti, e BeamNG.tech, orientato alla simulazione continua in tempo reale. Un'ulteriore limitazione riguarda la completezza della copertura dello standard OpenSCENARIO. Attualmente, la pipeline supporta un sottoinsieme delle primitive disponibili, focalizzandosi sugli elementi più frequentemente utilizzati negli scenari di testing automobilistico. Elementi avanzati quali le manovre cooperative

multi-veicolo o le condizioni ambientali dinamiche richiedono estensioni non ancora implementate. Dal punto di vista computazionale, la generazione di scenari ad alta complessità comporta tempi di elaborazione significativi, specialmente nella fase di interpolazione delle traiettorie e validazione fisica. Questo aspetto limita parzialmente l'applicabilità del sistema in contesti che richiedono generazione real-time di varianti scenariali.

6.4 Sviluppi Futuri

Il lavoro presentato apre diverse direzioni di ricerca e sviluppo che meritano approfondimento. Nel breve termine, l'integrazione di tecniche di intelligenza artificiale per la generazione automatica di scenari critici rappresenta un'evoluzione naturale del sistema. L'applicazione di algoritmi di ricerca adversarial e tecniche di ottimizzazione evolutiva potrebbe automatizzare l'identificazione di edge case e situazioni limite non facilmente individuabili attraverso approcci deterministici. Un'altra direzione promettente riguarda l'estensione del framework verso una piattaforma multi-simulatore. L'astrazione del layer di conversione attraverso un formato intermedio universale permetterebbe di supportare diversi ambienti di simulazione (CARLA, LGSVL, AirSim) con modifiche minime all'architettura, aumentando significativamente la portabilità degli scenari di test. Dal punto di vista delle metriche di valutazione, l'implementazione completa del diversity score attraverso tecniche di machine learning non supervisionato consentirebbe una caratterizzazione più sofisticata dello spazio degli scenari. L'integrazione di metriche basate su coverage criteria formali, derivati dall'analisi statica del codice dei sistemi di guida autonoma, potrebbe inoltre guidare la generazione verso scenari che massimizzano la copertura dei requisiti di sicurezza. A lungo termine, l'evoluzione del sistema verso un framework di validazione certificabile secondo standard automotive (ISO 26262, ISO/PAS 21448) costituirebbe un contributo fondamentale per l'adozione industriale. Questo richiederebbe l'integrazione di meccanismi di tracciabilità dei requisiti, generazione automatica di evidenze di test e supporto per analisi di safety assessment quantitative.

6.5 Considerazioni Finali

La validazione dei sistemi di guida autonoma rimane una delle sfide più complesse nell'ingegneria automotive moderna, richiedendo approcci innovativi che combinino rigore formale, scalabilità computazionale e rappresentatività fisica. Il lavoro presentato in questa tesi costituisce un passo concreto verso la sistematizzazione e automatizzazione del processo di testing attraverso simulazione, dimostrando la fattibilità tecnica di pipeline integrate per la gestione di scenari complessi. L'esperienza acquisita durante lo sviluppo ha evidenziato come la vera complessità non risieda tanto negli aspetti implementativi quanto nella necessità di conciliare paradigmi di modellazione eterogenei, ciascuno ottimizzato per specifici obiettivi. La soluzione proposta, pur con i limiti evidenziati, dimostra che tale conciliazione è possibile attraverso un'architettura modulare e estensibile. In conclusione, il framework sviluppato rappresenta una base solida per future ricerche nel campo del testing automatizzato per veicoli autonomi, contribuendo alla creazione di metodologie più robuste e affidabili per la validazione di sistemi safety-critical. L'auspicio è che questo lavoro possa stimolare ulteriori sviluppi verso l'obiettivo comune di rendere la mobilità autonoma non solo tecnicamente realizzabile, ma anche dimostrabilmente sicura e affidabile.

Bibliografia

- [1] S. Feng, F. Wotawa, and et al., “Machine learning-based test selection for simulation-based testing of self-driving cars software,” *Proceedings of the 42nd International Conference on Software Engineering*, 2020. (Citato alle pagine 1, 2, 3, 4, 5, 6, 7, 8, 9, 12, 14, 18, 20, 25 e 26)
- [2] X. Yang, J. Tao, M. T. Sn, I. Member, I. Qamar, A. Rahmani, M. Gyllenhammar, F. Wotawa, M. Nica, M. Behrisch, and H. Felbinger, “Critical scenarios for automated driving systems: Systematic mapping study,” *IEEE Transactions on Intelligent Transportation Systems*, 2021, systematic literature review on critical scenarios for ADS testing. (Citato alle pagine 1, 3, 4, 5, 6, 10, 11 e 12)
- [3] S. Riedmaier, T. Ponn, D. Ludwig, B. Schick, and F. Diermeyer, “Survey on scenario-based safety assessment of automated vehicles,” *IEEE Access*, vol. 8, pp. 87 456–87 477, 2020, comprehensive survey on scenario-based validation. (Citato alle pagine 1, 2, 3, 4, 5, 6, 9, 10, 11, 12, 13, 15, 16 e 17)
- [4] X. Zhang, J. Tao, K. Tan, M. Törngren, J. M. G. Sánchez, M. R. Ramli, X. Tao, M. Gyllenhammar, F. Wotawa, N. Mohan, M. Nica, and H. Felbinger, “Finding critical scenarios for automated driving systems: A systematic literature review,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 7, pp. 7177–7193, 2023, systematic literature review su identificazione di scenari critici;

- accessibile anche come arXiv:2110.08664 e IEEE document 9763411. (Citato alle pagine 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 14 e 17)
- [5] F. Bacchini, V. Cortellessa, S. Di Martino, D. Di Nucci, D. Di Pompeo, C. Gravino, and L. L. Strace, “Adas verification in co-simulation: Towards a meta-model for defining test scenarios,” in *Proceedings of the IEEE International Conference on Software Engineering*, 2021, co-simulation approach for ADAS validation. (Citato alle pagine 1, 2, 3, 4, 5, 6, 9, 10, 11, 12, 13 e 17)
- [6] S. Feng, F. Wotawa, and et al., “Cost-effective simulation-based test selection in self-driving cars software,” *Empirical Software Engineering*, vol. 26, no. 5, pp. 1–27, 2021. (Citato alle pagine 1, 2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 14, 15, 17, 18, 20 e 26)
- [7] C. Birchler, S. Khatiri, B. Bosshard, A. Gambi, and S. Panichella, “Machine learning-based test selection for simulation-based testing of self-driving cars software,” in *Proceedings of the 13th International Symposium on Search Based Software Engineering (SSBSE 2021)*, ser. Lecture Notes in Computer Science, vol. 12914. Springer, 2021, pp. 15–29, sDC-Scissor: ML-based test selection framework. (Citato alle pagine 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13, 14, 15 e 17)
- [8] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, “Carla: An open urban driving simulator,” pp. 1–16, 2017, original CARLA simulator paper. (Citato alle pagine 2, 3, 4, 6, 8, 10, 11, 12, 13, 14, 16 e 17)
- [9] A. Gambi, M. Mueller, and G. Fraser, “Automatically testing self-driving cars with search-based procedural content generation,” in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, 2019, pp. 318–328, automated scenario generation for ADS testing. (Citato alle pagine 2, 3, 4, 5, 6, 7, 8, 10, 11, 13, 14, 15, 16 e 17)
- [10] ASAM e.V., “Asam openscenario standard documentation,” <https://www.asam.net/standards/detail/openscenario/>, 2020, official ASAM OpenSCENARIO standard specification. (Citato alle pagine 2, 4, 5, 8, 12, 13 e 16)
- [11] R. Chandra, “Towards autonomous driving in dense, heterogeneous, and unstructured traffic,” Ph.D. dissertation, University of Maryland,

- College Park, 2022, PhD dissertation accessibile via ProQuest (ID 81d459d1ff6bbccbe468bfc6c69904ad); focus su guida autonoma in traffico denso e non strutturato. (Citato alle pagine 2, 3, 4, 5, 6 e 11)
- [12] A. Gambi, M. Müller, and G. Fraser, “Asfault: Testing self-driving car software using search-based procedural content generation,” in *Proceedings of the 41st IEEE/ACM International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 2019, pp. 318–328, beamNG-based procedural scenario generation for testing. (Citato alle pagine 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 e 17)
- [13] ASAM e.V., “Asam openscenario: A standard for description of dynamic content in driving simulators,” Association for Standardization of Automation and Measurement Systems, Tech. Rep., 2020, version 1.0. (Citato alle pagine 2, 4, 5, 8, 10, 12, 13, 14, 16 e 25)
- [14] —, “Asam openscenario: A standard for description of dynamic content in driving simulators,” Association for Standardization of Automation and Measurement Systems, Tech. Rep., 2020, version 1.0, <https://www.asam.net/standards/detail/openscenario/>. (Citato alle pagine 2, 4, 5, 8, 10, 13, 14 e 16)
- [15] C. Birchler, N. Ganz, S. Khatiri, A. Gambi, and S. Panichella, “Cost-effective simulation-based test selection in self-driving cars software with sdc-scissor,” in *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. Honolulu, HI, USA: IEEE, 2022, pp. 164–168, sDC-Scissor: Cost-effective test selection framework using machine learning for self-driving cars. (Citato alle pagine 3, 4 e 5)
- [16] Z. Tu, L. Niu, W. Fan, and T. Zhang, “Multi-modal traffic scenario generation for autonomous driving system testing,” *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, p. FSE078, jun 2025, multi-modal scenario generation from natural language and images for CARLA/LGSVL; bug detection capabilities. (Citato alle pagine 3, 4, 8 e 14)

-
- [17] CARLA Team, “Carla simulator,” 2024, <https://carla.org/>. (Citato alle pagine 3, 7, 10, 12, 13, 14, 16, 17 e 18)
- [18] BeamNG GmbH, “Beamng.tech,” 2024, <https://beamng.tech/>. (Citato alle pagine 3, 7, 10, 11, 15, 16, 17 e 18)
- [19] F. Basciani, V. Cortellessa, S. Di Martino, D. Di Nucci, D. Di Pompeo, C. Gravino, and L. L. L. Strace, “Adas verification in co-simulation: Towards a meta-model for defining test scenarios,” in *2023 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. Dublin, Ireland: IEEE, 2023, pp. 28–35, co-simulation approach for ADAS validation with meta-model for test scenarios. (Citato alle pagine 4, 5, 10, 11 e 12)
- [20] F. Kluck, F. Wotawa, and M. Nica, “Utilizing genetic algorithms for generating critical scenarios for testing autonomous driving functions,” in *Proceedings of the 6th IEEE International Conference on Artificial Intelligence Testing (AITest 2024)*. Shanghai, China: IEEE, 2024, pp. 73–80, iEEE document 11059240; search-based testing per generazione di scenari critici con algoritmi genetici. (Citato alle pagine 5, 6, 7, 8, 12, 13, 14 e 15)
- [21] —, “Utilizing genetic algorithms for generating critical scenarios for testing autonomous driving functions,” *e & i Elektrotechnik und Informationstechnik*, vol. 141, pp. 487–496, 2024, versione journal estesa; focus su design of experiments per hyperparameter tuning e confronto criticità/diversità. (Citato alle pagine 5, 12 e 14)
- [22] M. H. Moghadam, M. Borg, M. Saadatmand, S. J. Mousavirad, M. Bohlin, and B. Lisper, “Machine learning testing in an adas case study using simulation-integrated bio-inspired search-based testing,” *Journal of Software: Evolution and Process*, vol. 36, no. 5, p. e2591, 2024, bio-inspired search-based testing per ADAS integrato con simulazione. (Citato alle pagine 5, 7, 8 e 12)
- [23] SAE International, “Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles,” SAE International, Tech. Rep.

J3016_202104, 2021, standard SAE J3016 per i livelli di automazione veicolare.
(Citato alle pagine 10 e 11)